ALIS

0.1

Создано системой Doxygen 1.8.11

Оглавление

1 alis-c 1

2 Алфавитный указатель структур данных 3

2.1 Структуры данных 3

3 Список файлов 5

3.1 Файлы 5

4 Структуры данных 7

4.1 Структура io_header_t 7

4.1.1 Подробное описание 7

4.1.2 Поля 7

4.1.2.1 control 7

4.1.2.2 data_size 8

4.1.2.3 id 8

4.1.2.4 key_entry 8

4.1.2.5 main_entry 8

4.1.2.6 message_entry 8

4.1.2.7 msg_size 8

4.1.2.8 resources 8

4.1.2.9 stack_size 8

4.1.2.10 u1 9

4.1.2.11 u2 9

4.1.2.12 u4 9

4.1.2.13 u5 9

4.1.2.14 version . . . 9
4.2 Структура resource_table_t . . . 9
4.2.1 Подробное описание . . . 9
4.2.2 Поля . . . 10
4.2.2.1 anim_count . . . 10
4.2.2.2 anim_table . . . 10
4.2.2.3 image_count . . . 10
4.2.2.4 image_table . . . 10
4.2.2.5 sound_count . . . 10
4.2.2.6 sound_table . . . 10
4.3 Структура sound_channel_t . . . 10
4.3.1 Подробное описание . . . 11
4.3.2 Поля . . . 11
4.3.2.1 flags . . . 11
4.3.2.2 frequency . . . 11
4.3.2.3 frequency_gain . . . 11
4.3.2.4 gain . . . 11
4.3.2.5 length . . . 11
4.3.2.6 priority . . . 12
4.3.2.7 synth . . . 12
4.3.2.8 volume . . . 12
4.4 Структура sound_header_t . . . 12
4.4.1 Подробное описание . . . 12
4.4.2 Поля . . . 12
4.4.2.1 f1 . . . 12
4.4.2.2 f4 . . . 12
4.4.2.3 packed . . . 13
4.4.2.4 pad . . . 13
4.4.2.5 size . . . 13
4.4.2.6 type . . . 13

4.5 Структура sprite_t 13

4.5.1 Подробное описание 13

4.5.2 Поля 13

4.5.2.1 next 13

4.6 Структура thread_t 14

4.6.1 Подробное описание 14

4.6.2 Поля 14

4.6.2.1 call_sp 14

4.6.2.2 flags 15

4.6.2.3 frame_num 15

4.6.2.4 io_file 15

4.6.2.5 ip 15

4.6.2.6 memory 15

4.6.2.7 num_frames 15

4.6.2.8 running 15

4.6.2.9 sprites 16

4.7 Структура threads_table_t 16

4.7.1 Подробное описание 16

4.7.2 Поля 17

4.7.2.1 next 17

4.7.2.2 thread 17

5 Файлы 19

5.1 Файл blancpc.c . . . 19

5.1.1 Подробное описание . . . 20

5.1.2 Макросы . . . 20

5.1.2.1 BLANCPC . . . 20

5.1.2.2 BLANCPC_HEADER . . . 20

5.1.2.3 BLANCPC_SIZE . . . 20

5.1.2.4 BLANCPC_USIZE . . . 20

5.1.3 Функции . . . 21

5.1.3.1 load_blancpc() . . . 21

5.1.4 Переменные . . . 21

5.1.4.1 blancpc_buffer . . . 21

5.1.4.2 blancpc_data . . . 21

5.1.4.3 buffer . . . 21

5.2 blancpc.c . . . 22

5.3 Файл blancpc.h . . . 22

5.3.1 Функции . . . 23

5.3.1.1 load_blancpc() . . . 23

5.4 blancpc.h . . . 23

5.5 Файл file.c . . . 23

5.5.1 Подробное описание . . . 24

5.5.2 Функции . . . 24

5.5.2.1 file_close() . . . 24

5.5.2.2 file_exists(char *s) . . . 24

5.5.2.3 file_open(char *name, int mode) . . . 25

5.5.2.4 file_read(void *buf, int size) . . . 26

5.5.2.5 file_write(void *buf, int size) . . . 26

5.5.2.6 uppercase(char *s) . . . 26

5.5.3 Переменные . . . 27

5.5.3.1 file_name . . . 27

5.5.3.2 handle . . . 27
5.6 file.c . . . 27
5.7 Файл file.h . . . 28
5.7.1 Макросы . . . 28
5.7.1.1 FILE_APPEND . . . 28
5.7.1.2 FILE_CREATE . . . 29
5.7.2 Функции . . . 29
5.7.2.1 file_close() . . . 29
5.7.2.2 file_exists(char *s) . . . 29
5.7.2.3 file_open(char *file_name, int mode) . . . 29
5.7.2.4 file_read(void *buf, int size) . . . 30
5.8 file.h . . . 30
5.9 Файл io.c . . . 31
5.9.1 Подробное описание . . . 32
5.9.2 Перечисления . . . 32
5.9.2.1 anonymous enum . . . 32
5.9.3 Функции . . . 32
5.9.3.1 io_num_free_files() . . . 32
5.9.3.2 io_unpack_sound(byte *io) . . . 32
5.9.4 Переменные . . . 33
5.9.4.1 loaded_io_files . . . 33
5.9.4.2 sound_offset_table . . . 33
5.9.4.3 total_io_files . . . 33
5.10 io.c . . . 34
5.11 Файл io.h . . . 35
5.11.1 Функции . . . 35
5.11.1.1 io_num_free_files() . . . 35
5.11.1.2 io_unpack_sound(byte *io) . . . 36
5.12 io.h . . . 36
5.13 Файл key.c . . . 37

5.13.1 Подробное описание . . . 37
5.13.2 Функции . . . 38
5.13.2.1 get_key() . . . 38
5.13.2.2 get_key_mod() . . . 38
5.13.2.3 release_key(int scan) . . . 38
5.13.2.4 set_key(int scan, int sym, int mod) . . . 38
5.13.2.5 wait_for_key() . . . 39
5.13.3 Переменные . . . 39
5.13.3.1 key_mod . . . 39
5.13.3.2 key_symbol . . . 39
5.14 key.c . . . 40
5.15 Файл key.h . . . 41
5.15.1 Функции . . . 41
5.15.1.1 get_key() . . . 41
5.15.1.2 get_key_mod() . . . 41
5.15.1.3 release_key(int scan) . . . 41
5.15.1.4 set_key(int scan, int sym, int mod) . . . 42
5.15.1.5 wait_for_key() . . . 42
5.16 key.h . . . 43
5.17 Файл main.c . . . 43
5.17.1 Функции . . . 44
5.17.1.1 global_init() . . . 44
5.17.1.2 main() . . . 44
5.18 main.c . . . 44
5.19 Файл mouse.c . . . 45
5.19.1 Подробное описание . . . 45
5.19.2 Функции . . . 46
5.19.2.1 mouse_draw_cursor() . . . 46
5.19.2.2 mouse_event(int x, int y, int buttons) . . . 46
5.19.2.3 mouse_get(int *x, int *y, int *buttons) . . . 46

5.19.2.4 mouse_init() 46
5.19.3 Переменные 47
5.19.3.1 mouse_buttons 47
5.19.3.2 mouse_x 47
5.19.3.3 mouse_y 47
5.19.3.4 show_cursor 47
5.20 mouse.c 47
5.21 Файл mouse.h 48
5.21.1 Функции 48
5.21.1.1 mouse_draw_cursor() 48
5.21.1.2 mouse_event(int x, int y, int buttons) 48
5.21.1.3 mouse_get(int *x, int *y, int *buttons) 49
5.21.1.4 mouse_init() 49
5.22 mouse.h 49
5.23 Файл palette.c 49
5.23.1 Функции 49
5.23.1.1 palette_update() 49
5.24 palette.c 50
5.25 Файл palette.h 50
5.25.1 Функции 50
5.25.1.1 palette_update() 50
5.26 palette.h 50
5.27 Файл README.md 50
5.28 README.md 50
5.29 Файл sound.c 51
5.29.1 Подробное описание 52
5.29.2 Макросы 52
5.29.2.1 CHANNEL_OFF 52
5.29.2.2 MAX_VOLUME 52
5.29.2.3 NUM_CHANNELS 52

5.29.3 Функции . . . 52
5.29.3.1 sound_channel_off(sound_channel_t *ch) . . . 52
5.29.3.2 sound_find_channel_max_priority() . . . 53
5.29.3.3 sound_init() . . . 53
5.29.3.4 sound_process_channel(sound_channel_t *ch) . . . 53
5.29.3.5 sound_update() . . . 54
5.29.3.6 synth_play(sound_channel_t *ch) . . . 54
5.29.4 Переменные . . . 54
5.29.4.1 channel_num . . . 54
5.29.4.2 channel_num_max_priority . . . 54
5.29.4.3 current_channel . . . 55
5.29.4.4 sound_channels . . . 55
5.30 sound.c . . . 55
5.31 Файл sound.h . . . 56
5.31.1 Функции . . . 57
5.31.1.1 sound_init() . . . 57
5.31.1.2 sound_update() . . . 57
5.32 sound.h . . . 57
5.33 Файл sprites.c . . . 57
5.33.1 Подробное описание . . . 58
5.33.2 Функции . . . 59
5.33.2.1 sprites_free_num() . . . 59
5.33.2.2 sprites_set_translate(word *vec) . . . 59
5.33.2.3 sprites_translate(sprite_t *list, word *vec) . . . 59
5.33.3 Переменные . . . 59
5.33.3.1 free_sprites . . . 59
5.34 sprites.c . . . 59
5.35 Файл sprites.h . . . 60
5.35.1 Функции . . . 61
5.35.1.1 sprites_free_num() . . . 61

5.35.1.2 sprites_set_translate(word *vec) . . . 61
5.35.1.3 sprites_translate(sprite_t *list, word *vec) . . . 61
5.36 sprites.h . . . 61
5.37 Файл text.c . . . 61
5.37.1 Функции . . . 62
5.37.1.1 set_cursor_pos(int x, int y) . . . 62
5.37.2 Переменные . . . 62
5.37.2.1 cursor_x . . . 62
5.37.2.2 cursor_y . . . 62
5.38 text.c . . . 62
5.39 Файл text.h . . . 62
5.39.1 Функции . . . 63
5.39.1.1 set_cursor_pos(int x, int y) . . . 63
5.40 text.h . . . 63
5.41 Файл threads.c . . . 63
5.41.1 Подробное описание . . . 64
5.41.2 Функции . . . 64
5.41.2.1 threads_free_num() . . . 64
5.41.2.2 threads_init() . . . 64
5.41.2.3 threads_run() . . . 65
5.41.3 Переменные . . . 65
5.41.3.1 current_thread . . . 65
5.41.3.2 loaded_threads . . . 65
5.41.3.3 max_threads . . . 66
5.41.3.4 no_save . . . 66
5.41.3.5 thread_flag . . . 66
5.41.3.6 threads_list . . . 66
5.41.3.7 threads_list_pos . . . 66
5.41.3.8 threads_table . . . 66
5.42 threads.c . . . 67

5.43 Файл threads.h . . . 67
5.43.1 Макросы . . . 69
5.43.1.1 THREAD_MSG . . . 69
5.43.1.2 THREAD_NOMSG . . . 69
5.43.1.3 THREAD_NOSTART3 . . . 69
5.43.2 Функции . . . 69
5.43.2.1 threads_free_num() . . . 69
5.43.2.2 threads_init() . . . 69
5.43.2.3 threads_run() . . . 70
5.44 threads.h . . . 70
5.45 Файл timer.c . . . 71
5.45.1 Функции . . . 71
5.45.1.1 timer_update() . . . 71
5.45.2 Переменные . . . 72
5.45.2.1 frame_num . . . 72
5.46 timer.c . . . 72
5.47 Файл timer.h . . . 72
5.47.1 Функции . . . 72
5.47.1.1 timer_update() . . . 72
5.48 timer.h . . . 73
5.49 Файл types.h . . . 73
5.49.1 Типы . . . 73
5.49.1.1 bool . . . 73
5.49.1.2 byte . . . 73
5.49.1.3 dword . . . 73
5.49.1.4 word . . . 73
5.50 types.h . . . 74
5.51 Файл video.c . . . 74
5.51.1 Подробное описание . . . 75
5.51.2 Макросы . . . 75

5.51.2.1 FPS . . . 75
5.51.2.2 SCREEN_HEIGHT . . . 75
5.51.2.3 SCREEN_WIDTH . . . 75
5.51.2.4 WINDOW_HEIGHT . . . 75
5.51.2.5 WINDOW_WIDTH . . . 75
5.51.3 Функции . . . 76
5.51.3.1 video_close() . . . 76
5.51.3.2 video_init() . . . 76
5.51.3.3 video_update() . . . 76
5.51.3.4 video_update_events() . . . 77
5.51.4 Переменные . . . 77
5.51.4.1 current_time . . . 77
5.51.4.2 renderer . . . 77
5.51.4.3 screen . . . 77
5.51.4.4 time_step . . . 77
5.51.4.5 video_buffer . . . 78
5.51.4.6 window . . . 78
5.52 video.c . . . 78
5.53 Файл video.h . . . 79
5.53.1 Функции . . . 79
5.53.1.1 video_init() . . . 79
5.53.1.2 video_update() . . . 80
5.53.1.3 video_update_events() . . . 80
5.54 video.h . . . 81
5.55 Файл view.c . . . 81
5.55.1 Функции . . . 81
5.55.1.1 view_update() . . . 81
5.56 view.c . . . 81
5.57 Файл view.h . . . 81
5.57.1 Функции . . . 82

5.57.1.1 view_update() . . . 82
5.58 view.h . . . 82
5.59 Файл vm.c . . . 82
5.59.1 Подробное описание . . . 83
5.59.2 Макросы . . . 83
5.59.2.1 CPU_SPEED . . . 83
5.59.2.2 FREE_MEM . . . 83
5.59.2.3 HARDWARE_FLAG . . . 83
5.59.2.4 MAX_STR . . . 84
5.59.2.5 VIDEO_PORT . . . 84
5.59.3 Функции . . . 84
5.59.3.1 get_free() . . . 84
5.59.3.2 vm_init() . . . 84
5.59.3.3 vm_run(byte *ip) . . . 85
5.59.4 Переменные . . . 85
5.59.4.1 accum . . . 85
5.59.4.2 get_string . . . 85
5.59.4.3 get_string_buf . . . 85
5.59.4.4 store_string . . . 85
5.59.4.5 store_string_buf . . . 85
5.59.4.6 text_string . . . 85
5.59.4.7 text_string_buf . . . 86
5.60 vm.c . . . 86
5.61 Файл vm.h . . . 87
5.61.1 Функции . . . 87
5.61.1.1 vm_init() . . . 87
5.61.1.2 vm_run(byte *ip) . . . 87
5.62 vm.h . . . 87

Алфавитный указатель 89

Глава 1

alis-c

Глава 2

Алфавитный указатель структур данных

2.1 Структуры данных

Структуры данных с их кратким описанием.

io_header_t 7
resource_table_t
таблица ресурсов 9
sound_channel_t 10
sound_header_t 12
sprite_t 13
thread_t
структура потока 14
threads_table_t
запись таблицы потоков 16

Глава 3

Список файлов

3.1 Файлы

Полный список файлов.

blancpc.c
Сценарий - пустой компьютер . . . 19
blancpc.h . . . 22
file.c
Работа с файлами . . . 23
file.h . . . 28
io.c
Работа с IO файлом, ресурсы: графика, звук . . . 31
io.h . . . 35
key.c
Работа с клавиатурой . . . 37
key.h . . . 41
main.c . . . 43
mouse.c
Управление курсором мыши . . . 45
mouse.h . . . 48
palette.c . . . 49
palette.h . . . 50
sound.c
Модуль обработки звуков (синтезатор + оцифрованный звук) . . . 51
sound.h . . . 56
sprites.c
Модуль для работы со спрайтами . . . 57
sprites.h . . . 60
text.c . . . 61
text.h . . . 62
threads.c
Модуль работы с потоками . . . 63
threads.h . . . 67
timer.c . . . 71
timer.h . . . 72
types.h . . . 73
video.c
Модуль для работы с графикой SDL: окно, обработка событий, вывод графики . . . 74
video.h . . . 79

view.c 81
view.h 81
vm.c
Интерпретатор виртуальной машины 82
vm.h 87

Глава 4

Структуры данных

4.1 Структура io_header_t

#include <io.h>

Поля данных

- word id
- word main_entry
- byte control
- byte version
- word key_entry
- byte u1
- byte u2
- word message_entry
- byte u4
- byte u5
- dword resources
- word stack_size
- word data_size
- word msg_size

4.1.1 Подробное описание

См. определение в файле io.h строка

4.1.2 Поля

4.1.2.1 byte io_header_t::control

См. определение в файле io.h строка

4.1.2.2 word io_header_t::data_size

размер сегмента данных

См. определение в файле io.h строка

4.1.2.3 word io_header_t::id

номер сценария

См. определение в файле io.h строка

4.1.2.4 word io_header_t::key_entry

обработка нажатий клавиш

См. определение в файле io.h строка

4.1.2.5 word io_header_t::main_entry

адрес начала сценария

См. определение в файле io.h строка

4.1.2.6 word io_header_t::message_entry

обработка сообщений

См. определение в файле io.h строка

4.1.2.7 word io_header_t::msg_size

размер стека сообщений

См. определение в файле io.h строка

4.1.2.8 dword io_header_t::resources

адрес таблицы ресурсов

См. определение в файле io.h строка

4.1.2.9 word io_header_t::stack_size

размер стека вызовов

См. определение в файле io.h строка

4.1.2.10 byte io_header_t::u1

См. определение в файле io.h строка

4.1.2.11 byte io_header_t::u2

См. определение в файле io.h строка

4.1.2.12 byte io_header_t::u4

См. определение в файле io.h строка

4.1.2.13 byte io_header_t::u5

См. определение в файле io.h строка

4.1.2.14 byte io_header_t::version

версия сценария

См. определение в файле io.h строка

Объявления и описания членов структуры находятся в файле:

- io.h

4.2 Структура resource_table_t

таблица ресурсов

Поля данных

- dword image_table
- word image_count
- dword anim_table
- word anim_count
- dword sound_table
- word sound_count

4.2.1 Подробное описание

таблица ресурсов

См. определение в файле io.c строка

4.2.2 Поля

4.2.2.1 word resource_table_t::anim_count

См. определение в файле io.c строка

4.2.2.2 dword resource_table_t::anim_table

См. определение в файле io.c строка

4.2.2.3 word resource_table_t::image_count

число изображений

См. определение в файле io.c строка

4.2.2.4 dword resource_table_t::image_table

смещение таблицы изображений

См. определение в файле io.c строка

4.2.2.5 word resource_table_t::sound_count

число звуков

См. определение в файле io.c строка

4.2.2.6 dword resource_table_t::sound_table

смещение таблицы звуков

См. определение в файле io.c строка

Объявления и описания членов структуры находятся в файле:

- io.c

4.3 Структура sound_channel_t

#include <sound.h>

Поля данных

- byte flags
- char priority
- char synth
- word length
- short volume
- short gain
- short frequency
- short frequency_gain

4.3.1 Подробное описание

См. определение в файле sound.h строка

4.3.2 Поля

4.3.2.1 byte sound_channel_t::flags

флаги канала

См. определение в файле sound.h строка

4.3.2.2 short sound_channel_t::frequency

частота ноты

См. определение в файле sound.h строка

4.3.2.3 short sound_channel_t::frequency_gain

смещение частоты ноты во времени

См. определение в файле sound.h строка

4.3.2.4 short sound_channel_t::gain

смещение громкости (усиление и затухание звука во времени) ноты

См. определение в файле sound.h строка

4.3.2.5 word sound_channel_t::length

число кадров - длительность звучания ноты

См. определение в файле sound.h строка

4.3.2.6 char sound_channel_t::priority

приоритет канала, звучит только канал с максимальным приоритетом

См. определение в файле sound.h строка

4.3.2.7 char sound_channel_t::synth

если больше 0, то канал синтезатора

См. определение в файле sound.h строка

4.3.2.8 short sound_channel_t::volume

громкость звучания ноты

См. определение в файле sound.h строка

Объявления и описания членов структуры находятся в файле:

- sound.h

4.4 Структура sound_header_t

Поля данных

- byte type
- byte f1
- word size
- word f4
- byte packed
- byte pad [9]

4.4.1 Подробное описание

См. определение в файле io.c строка

4.4.2 Поля

4.4.2.1 byte sound_header_t::f1

См. определение в файле io.c строка

4.4.2.2 word sound_header_t::f4

См. определение в файле io.c строка

4.4.2.3 byte sound_header_t::packed

если 1 то звук упакован

См. определение в файле io.c строка

4.4.2.4 byte sound_header_t::pad[9]

См. определение в файле io.c строка

4.4.2.5 word sound_header_t::size

размер данных

См. определение в файле io.c строка

4.4.2.6 byte sound_header_t::type

тип звука

См. определение в файле io.c строка

Объявления и описания членов структуры находятся в файле:

- io.c

4.5 Структура sprite_t

#include <sprites.h>

Поля данных

- struct sprite_s * next

4.5.1 Подробное описание

См. определение в файле sprites.h строка

4.5.2 Поля

4.5.2.1 struct sprite_s* sprite_t::next

указатель на следующий в списке (список потока или список свободных ячеек)

См. определение в файле sprites.h строка

Объявления и описания членов структуры находятся в файле:

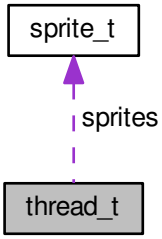
- sprites.h

4.6 Структура thread_t

структура потока

#include <threads.h>

Граф связей класса thread_t:



Поля данных

- byte flags
- char running
- int num_frames
- int frame_num
- byte * io_file
- byte * memory
- word ip
- word call_sp
- sprite_t * sprites

4.6.1 Подробное описание

структура потока

См. определение в файле threads.h строка

4.6.2 Поля

4.6.2.1 word thread_t::call_sp

указатель стека вызовов

См. определение в файле threads.h строка

4.6.2.2 byte thread_t::flags

флаги потока

См. определение в файле threads.h строка

4.6.2.3 int thread_t::frame_num

счетчик кадров

См. определение в файле threads.h строка

4.6.2.4 byte* thread_t::io_file

загруженный образ io файла

См. определение в файле threads.h строка

4.6.2.5 word thread_t::ip

указатель команд

См. определение в файле threads.h строка

4.6.2.6 byte* thread_t::memory

память потока

См. определение в файле threads.h строка

4.6.2.7 int thread_t::num_frames

число кадров за которое поток работает 1 раз

См. определение в файле threads.h строка

4.6.2.8 char thread_t::running

если 0 - поток не работает

См. определение в файле threads.h строка

4.6.2.9 sprite_t* thread_t::sprites

список спрайтов потока

См. определение в файле threads.h строка

Объявления и описания членов структуры находятся в файле:

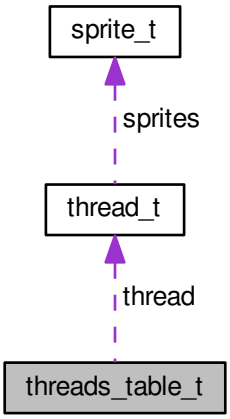
- threads.h

4.7 Структура threads_table_t

запись таблицы потоков

#include <threads.h>

Граф связей класса threads_table_t:



Поля данных

- thread_t * thread
- struct threads_table_s * next

4.7.1 Подробное описание

запись таблицы потоков

См. определение в файле threads.h строка

4.7.2 Поля

4.7.2.1 struct threads_table_s* threads_table_t::next

следующий поток в списке в запущенных

См. определение в файле threads.h строка

4.7.2.2 thread_t* threads_table_t::thread

указатель на поток

См. определение в файле threads.h строка

Объявления и описания членов структуры находятся в файле:

- threads.h

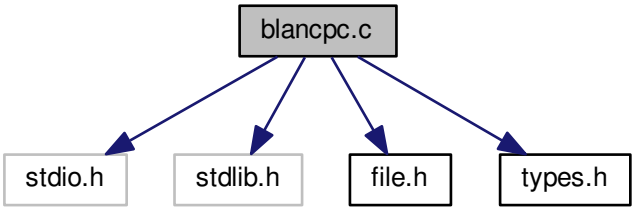
Глава 5

Файлы

5.1 Файл blancpc.c

Сценарий - пустой компьютер

#include <stdio.h>
#include <stdlib.h>
#include "file.h"
#include "types.h"
Граф включаемых заголовочных файлов для blancpc.c:



Макросы

- #define BLANCPC "blancpc.io"
- #define BLANCPC_SIZE 2500
- #define BLANCPC_USIZE 2500
- #define BLANCPC_HEADER 54

Функции

- void load_blancpc ()

Переменные

- byte buffer [BLANCPC_SIZE]
- byte blancpc_buffer [2560]
- byte * blancpc_data

5.1.1 Подробное описание

Сценарий - пустой компьютер

Автор

alex <alex>

Дата

Sun Jan 30 12:30:00 2022

См. определение в файле blancpc.c

5.1.2 Макросы

5.1.2.1 #define BLANCPC "blancpc.io"

имя файла сценария

См. определение в файле blancpc.c строка

5.1.2.2 #define BLANCPC_HEADER 54

размер заголовка - незакодированные данные

См. определение в файле blancpc.c строка

5.1.2.3 #define BLANCPC_SIZE 2500

запакованный размер

См. определение в файле blancpc.c строка

5.1.2.4 #define BLANCPC_USIZE 2500

разпакованный размер

См. определение в файле blancpc.c строка

5.1.3 Функции

5.1.3.1 void load_blancpc ( )

См. определение в файле blancpc.c строка

```
00026 {
00027   byte *src;
00028   byte *dst;
00029   FILE *f;
00030   file_open(BLANCPC, 2);
00031   file_read(buffer, BLANCPC_SIZE);
00032   file_close();
00033   blancpc_data = buffer + BLANCPC_HEADER;
00034   src = blancpc_data;
00035   dst = blancpc_buffer + BLANCPC_HEADER;
00036   for (int i = 0; i < 116; i++)
00037     *dst++ = *src++;
00038   for (int i = 0; i < 2444; i++) {
00039     byte b = *src++;
00040     if (b) {
00041       b -= 32;
00042       byte b2 = (b >> 1) + (b & 0x80);
00043       b = b2;
00044       b2 = (b >> 1) + (b & 0x80);
00045       b = b2 + 32;
00046     }
00047     *dst++ = b;
00048   }
00049 }
```

5.1.4 Переменные

5.1.4.1 byte blancpc_buffer[2560]

разшифрованный

См. определение в файле blancpc.c строка

5.1.4.2 byte* blancpc_data

См. определение в файле blancpc.c строка

5.1.4.3 byte buffer[BLANCPC_SIZE]

буфер сценария - зашифрованный

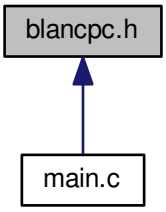
См. определение в файле blancpc.c строка

5.2 blancpc.c

```
#include <stdio.h>
#include <stdlib.h>
#include "file.h"
#include "types.h"

#define BLANCPC "blancpc.io"
#define BLANCPC_SIZE 2500
#define BLANCPC_USIZE 2500
#define BLANCPC_HEADER 54
byte buffer[BLANCPC_SIZE];
byte blancpc_buffer[2560];
byte *blancpc_data;

void load_blancpc()
{
  byte *src;
  byte *dst;
  FILE *f;
  file_open(BLANCPC, 2);
  file_read(buffer, BLANCPC_SIZE);
  file_close();
  blancpc_data = buffer + BLANCPC_HEADER;
  src = blancpc_data;
  dst = blancpc_buffer + BLANCPC_HEADER;
  for (int i = 0; i < 116; i++)
    *dst++ = *src++;
  for (int i = 0; i < 2444; i++) {
    byte b = *src++;
    if (b) {
      b -= 32;
      byte b2 = (b >> 1) + (b & 0x80);
      b = b2;
      b2 = (b >> 1) + (b & 0x80);
      b = b2 + 32;
    }
    *dst++ = b;
  }
}
```

5.3 Файл blancpc.h

Граф файлов, в которые включается этот файл:



Функции

- void load_blancpc ()

5.3.1 Функции

5.3.1.1 void load_blancpc ( )

См. определение в файле blancpc.c строка

```
00026 {
00027   byte *src;
00028   byte *dst;
00029   FILE *f;
00030   file_open(BLANCPC, 2);
00031   file_read(buffer, BLANCPC_SIZE);
00032   file_close();
00033   blancpc_data = buffer + BLANCPC_HEADER;
00034   src = blancpc_data;
00035   dst = blancpc_buffer + BLANCPC_HEADER;
00036   for (int i = 0; i < 116; i++)
00037     *dst++ = *src++;
00038   for (int i = 0; i < 2444; i++) {
00039     byte b = *src++;
00040     if (b) {
00041       b -= 32;
00042       byte b2 = (b >> 1) + (b & 0x80);
00043       b = b2;
00044       b2 = (b >> 1) + (b & 0x80);
00045       b = b2 + 32;
00046     }
00047     *dst++ = b;
00048   }
00049 }
```

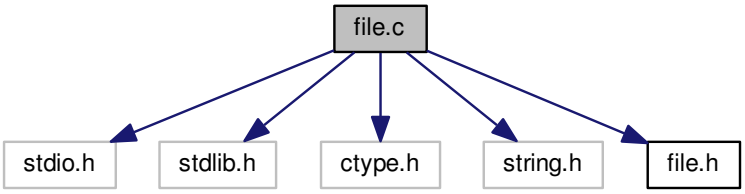
5.4 blancpc.h

```
00001 void load_blancpc();
```

5.5 Файл file.c

Работа с файлами

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include "file.h"
Граф включаемых заголовочных файлов для file.c:



Функции

- void uppercase (char *s)

  преобразование строки в верхний регистр
- void file_open (char *name, int mode)
- void file_read (void *buf, int size)
- void file_write (void *buf, int size)
- void file_close ()
- int file_exists (char *s)

Переменные

- FILE * handle
- char file_name [20]

5.5.1 Подробное описание

Работа с файлами

Автор

alex <alex>

Дата

Sun Jan 30 14:07:11 2022

См. определение в файле file.c

5.5.2 Функции

5.5.2.1 void file_close ( )

Закрытие текущего файла

См. определение в файле file.c строка

```
00095 {
00096   fclose(handle);
00097 }
```

5.5.2.2 int file_exists (char * s)

Проверка на существование файла. Имя файла преобразуется в верхний регистр при необходимости

Аргументы

s	имя файла

Возвращает

1 если файл существует, иначе 0

См. определение в файле file.c строка

```
00108 {
00109   FILE *f = fopen(s, "rb");
00110   if (!f) {
00111     uppercase(s);
00112     f = fopen(s, "rb");
00113     if (!f)
00114       return 0;
00115   }
00116   fclose(f);
00117   return 1;
00118 }
```

5.5.2.3 void file_open (char * name, int mode)

Открытие или создание файла. Пытается открыть файл с именем в верхнем регистре

Аргументы

file_name	путь к файлу
mode	режим открытия

Возвращает

указатель на файл

См. определение в файле file.c строка

```
00038 {
00039   char rw[] = "r+b";
00040   char cr[] = "w+b";
00041   char ap[] = "a+b";
00042   char *m;
00043   strcpy(file_name, name);
00044   if (mode & FILE_CREATE)
00045     m = cr;
00046   else if (mode & FILE_APPEND)
00047     m = ap;
00048   else
00049     m = rw;
00050   handle = fopen(file_name, m);
00051   if (!handle) {
00052     uppercase(file_name);
00053     handle = fopen(file_name, m);
00054     if (!handle) {
00055       fprintf(stderr, "File: %s not found\n", file_name);
00056       exit(1);
00057     }
00058   }
00059 }
```

5.5.2.4 void file_read (void * buf, int size)

Чтение из файла с обработкой ошибок

Аргументы

buf	адрес буфера куда происходит чтение
size	размер буфера

См. определение в файле file.c строка

```
00068 {
00069   int c = fread(buf, size, 1, handle);
00070   if (c < 0) {
00071     fprintf(stderr, "File: read error size = %d, read bytes: %d\n", size, c);
00072     exit(1);
00073   }
00074 }
```

5.5.2.5 void file_write (void * buf, int size)

Запись в файл с обработкой ошибок

Аргументы

buf	адрес буфера который записывается
size	размер буфера

См. определение в файле file.c строка

```
00083 {
00084   int c = fwrite(buf, size, 1, handle);
00085   if (c < 0) {
00086     fprintf(stderr, "File: write error size = %d, read bytes: %d\n", size, c);
00087     exit(1);
00088   }
00089 }
```

5.5.2.6 void uppercase (char * s)

преобразование строки в верхний регистр

См. определение в файле file.c строка

```
00021 {
00022     char *c = s;
00023     while (*c) {
00024       *c = toupper(*c);
00025       c++;
00026     }
00027 }
```

5.5.3 Переменные

5.5.3.1 char file_name[20]

имя файла

См. определение в файле file.c строка

5.5.3.2 FILE* handle

текущий файл с которым идет работа

См. определение в файле file.c строка

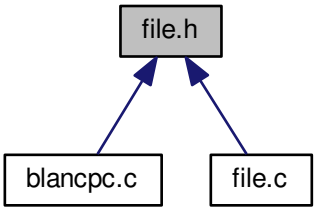
5.6 file.c

```
00001
00010 #include <stdio.h>
00011 #include <stdlib.h>
00012 #include <ctype.h>
00013 #include <string.h>
00014 #include "file.h"
00015
00016 FILE *handle;
00017 char file_name[20];
00019 void uppercase(char *s)
00021 {
00022     char *c = s;
00023     while (*c) {
00024       *c = toupper(*c);
00025       c++;
00026     }
00027 }
00028
00037 void file_open(char *name, int mode)
00038 {
00039   char rw[] = "r+b";
00040   char cr[] = "w+b";
00041   char ap[] = "a+b";
00042   char *m;
00043   strcpy(file_name, name);
00044   if (mode & FILE_CREATE)
00045     m = cr;
00046   else if (mode & FILE_APPEND)
00047     m = ap;
00048   else
00049     m = rw;
00050   handle = fopen(file_name, m);
00051   if (!handle) {
00052     uppercase(file_name);
00053     handle = fopen(file_name, m);
00054     if (!handle) {
00055       fprintf(stderr, "File: %s not found\n", file_name);
00056       exit(1);
00057     }
00058   }
00059 }
00060
00067 void file_read(void *buf, int size)
00068 {
00069   int c = fread(buf, size, 1, handle);
00070   if (c < 0) {
00071     fprintf(stderr, "File: read error size = %d, read bytes: %d\n", size, c);
00072     exit(1);
00073   }
00074 }
00075
00082 void file_write(void *buf, int size)
00083 {
00084   int c = fwrite(buf, size, 1, handle);
00085   if (c < 0) {
00086     fprintf(stderr, "File: write error size = %d, read bytes: %d\n", size, c);
```

```
    exit(1);
  }
}

void file_close()
{
  fclose(handle);
}

int file_exists(char *s)
{
  FILE *f = fopen(s, "rb");
  if (!f) {
    uppercase(s);
    f = fopen(s, "rb");
    if (!f)
      return 0;
  }
  fclose(f);
  return 1;
}
```

5.7 Файл file.h

Граф файлов, в которые включается этот файл:



Макросы

- #define FILE_CREATE 0x200
- #define FILE_APPEND 0x800

Функции

- void file_open (char *file_name, int mode)
- void file_read (void *buf, int size)
- void file_close ()
- int file_exists (char *s)

5.7.1 Макросы

5.7.1.1 #define FILE_APPEND 0x800

открытие на добавление файла

См. определение в файле file.h строка

5.7.1.2 #define FILE_CREATE 0x200

создание файла

См. определение в файле file.h строка

5.7.2 Функции

5.7.2.1 void file_close ( )

Закрытие текущего файла

См. определение в файле file.c строка

```
00095 {
00096   fclose(handle);
00097 }
```

5.7.2.2 int file_exists (char * s)

Проверка на существование файла. Имя файла преобразуется в верхний регистр при необходимости

Аргументы

s	имя файла

Возвращает

1 если файл существует, иначе 0

См. определение в файле file.c строка

```
00108 {
00109   FILE *f = fopen(s, "rb");
00110   if (!f) {
00111     uppercase(s);
00112     f = fopen(s, "rb");
00113     if (!f)
00114       return 0;
00115   }
00116   fclose(f);
00117   return 1;
00118 }
```

5.7.2.3 void file_open (char * name, int mode)

Открытие или создание файла. Пытается открыть файл с именем в верхнем регистре

Аргументы

file_name	путь к файлу
mode	режим открытия

Возвращает

указатель на файл

См. определение в файле file.c строка

```
00038 {
00039   char rw[] = "r+b";
00040   char cr[] = "w+b";
00041   char ap[] = "a+b";
00042   char *m;
00043   strcpy(file_name, name);
00044   if (mode & FILE_CREATE)
00045     m = cr;
00046   else if (mode & FILE_APPEND)
00047     m = ap;
00048   else
00049     m = rw;
00050   handle = fopen(file_name, m);
00051   if (!handle) {
00052     uppercase(file_name);
00053     handle = fopen(file_name, m);
00054     if (!handle) {
00055       fprintf(stderr, "File: %s not found\n", file_name);
00056       exit(1);
00057     }
00058   }
00059 }
```

5.7.2.4 void file_read (void * buf, int size)

Чтение из файла с обработкой ошибок

Аргументы

buf	адрес буфера куда происходит чтение
size	размер буфера

См. определение в файле file.c строка

```
00068 {
00069   int c = fread(buf, size, 1, handle);
00070   if (c < 0) {
00071     fprintf(stderr, "File: read error size = %d, read bytes: %d\n", size, c);
00072     exit(1);
00073   }
00074 }
```

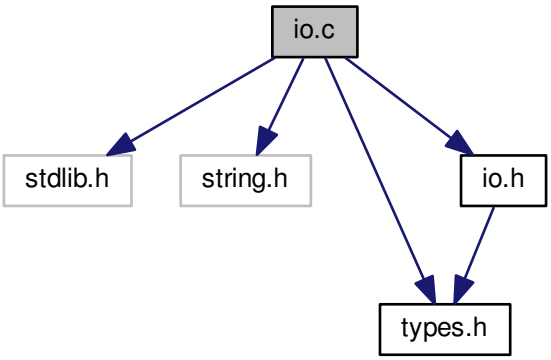
5.8 file.h

```
00001 #define FILE_CREATE 0x200
00002 #define FILE_APPEND 0x800
00004 void file_open(char *file_name, int mode);
00005 void file_read(void *buf, int size);
00006 void file_close();
00007 int file_exists(char *s);
```

5.9 Файл io.c

Работа с IO файлом, ресурсы: графика, звук

#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "io.h"
Граф включаемых заголовочных файлов для io.c:



Структуры данных

- struct resource_table_t

  таблица ресурсов
- struct sound_header_t

Перечисления

- enum { SOUND_TYPE1 = 1, SOUND_TYPE2 = 2 }

Функции

- void io_unpack_sound (byte *io)
- int io_num_free_files ()

Переменные

- int total_io_files
- int loaded_io_files
- char sound_offset_table [] = {-34, -21, -13, -8, -5, -3, -2, -1, 1, 2, 3, 5, 8, 13, 21, 34}

  таблица смещений в данных звука

5.9.1 Подробное описание

Работа с IO файлом, ресурсы: графика, звук

Автор

alex <alex>

Дата

Tue Apr 5 07:30:44 2022

См. определение в файле io.c

5.9.2 Перечисления

5.9.2.1 anonymous enum

Элементы перечислений

SOUND_TYPE1
SOUND_TYPE2

См. определение в файле io.c строка

```
00026     {
00027   SOUND_TYPE1 = 1,
00028   SOUND_TYPE2 = 2,
00029 };
```

5.9.3 Функции

5.9.3.1 int io_num_free_files ( )

Возвращает число свободных ячеек из таблицы загруженных io файлов

См. определение в файле io.c строка

```
00090 {
00091   return total_io_files - loaded_io_files;
00092 }
```

5.9.3.2 void io_unpack_sound (byte * io)

Распаковка звуковых данных в io файле Звуковые данные упакованы как закодированные (по 4 бит) смещения

Аргументы

io	буфер данных io файла

См. определение в файле io.c строка

```
00051 {
00052   io_header_t *h = (io_header_t *)io;
00053   resource_table_t *res = (resource_table_t *)(io + h->
    resources);
00054   dword *tab = (dword *)((byte *)res + res->sound_table);
00055   sound_header_t *data;
00056   byte *temp;
00057   byte *src;
00058   byte *dst;
00059   char ofs;
00060   int count;
00061   for (int i = 0; i < res->sound_count; i++) {
00062     data = (sound_header_t *)((byte *)tab + *tab);
00063     if (data->type == SOUND_TYPE2 || data->type ==
    SOUND_TYPE1)
00064       if (data->packed == 1) {
00065     count = data->size - sizeof(sound_header_t) >> 1;
00066     temp = malloc(count);
00067     memcpy(temp, data + 1, count);
00068     src = temp;
00069     dst = (byte *)(data + 1);
00070     ofs = *(char *)src++;
00071     *dst++ = ofs;
00072     for (int j = 0; j < count - 1; j++) {
00073       ofs += sound_offset_table[*src >> 4];
00074       *dst++ = ofs;
00075       ofs += sound_offset_table[*src++ & 0xf];
00076       *dst++ = ofs;
00077     }
00078     dst[-1] = dst[-2] = 0;
00079     data->size -= 2;
00080     free(temp);
00081       }
00082     ++tab;
00083   }
00084 }
```

5.9.4 Переменные

5.9.4.1 int loaded_io_files

число загруженных io файлов

См. определение в файле io.c строка

5.9.4.2 char sound_offset_table[] = {-34, -21, -13, -8, -5, -3, -2, -1, 1, 2, 3, 5, 8, 13, 21, 34}

таблица смещений в данных звука

См. определение в файле io.c строка

5.9.4.3 int total_io_files

максимальное число загруженных io файлов

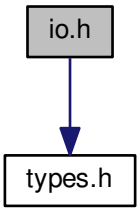
См. определение в файле io.c строка

5.10 io.c

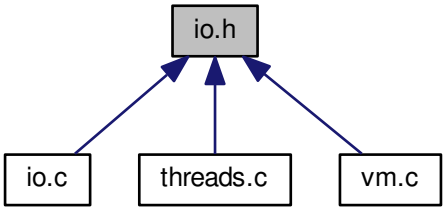
```
#include <stdlib.h>
#include <string.h>

#include "types.h"
#include "io.h"

typedef struct {
  dword image_table;
  word image_count;
  dword anim_table;
  word anim_count;
  dword sound_table;
  word sound_count;
} resource_table_t;

enum {
  SOUND_TYPE1 = 1,
  SOUND_TYPE2 = 2,
};

typedef struct {
  byte type;
  byte f1;
  word size;
  word f4;
  byte packed;
  byte pad[9];
} sound_header_t;

int total_io_files;
int loaded_io_files;
char sound_offset_table[] = {-34, -21, -13, -8, -5, -3, -2, -1, 1, 2, 3, 5, 8, 13, 21, 34
    };

void io_unpack_sound(byte *io)
{
  io_header_t *h = (io_header_t *)io;
  resource_table_t *res = (resource_table_t *)(io + h->
    resources);
  dword *tab = (dword *)((byte *)res + res->sound_table);
  sound_header_t *data;
  byte *temp;
  byte *src;
  byte *dst;
  char ofs;
  int count;
  for (int i = 0; i < res->sound_count; i++) {
    data = (sound_header_t *)((byte *)tab + *tab);
    if (data->type == SOUND_TYPE2 || data->type ==
    SOUND_TYPE1)
      if (data->packed == 1) {
    count = data->size - sizeof(sound_header_t) >> 1;
    temp = malloc(count);
    memcpy(temp, data + 1, count);
    src = temp;
    dst = (byte *)(data + 1);
    ofs = *(char *)src++;
    *dst++ = ofs;
    for (int j = 0; j < count - 1; j++) {
      ofs += sound_offset_table[*src >> 4];
      *dst++ = ofs;
      ofs += sound_offset_table[*src++ & 0xf];
      *dst++ = ofs;
    }
    dst[-1] = dst[-2] = 0;
    data->size -= 2;
    free(temp);
      }
    ++tab;
  }
}

int io_num_free_files()
{
  return total_io_files - loaded_io_files;
}
```

5.11 Файл io.h

#include "types.h"
Граф включаемых заголовочных файлов для io.h:



Граф файлов, в которые включается этот файл:



Структуры данных

- struct io_header_t

Функции

- void io_unpack_sound (byte *io)
- int io_num_free_files ()

5.11.1 Функции

5.11.1.1 int io_num_free_files ( )

Возвращает число свободных ячеек из таблицы загруженных io файлов

См. определение в файле io.c строка

```
00090 {
00091   return total_io_files - loaded_io_files;
00092 }
```

5.11.1.2 void io_unpack_sound (byte * io)

Распаковка звуковых данных в io файле Звуковые данные упакованы как закодированные (по 4 бит) смещения

Аргументы

io	буфер данных io файла

См. определение в файле io.c строка

```
00051 {
00052   io_header_t *h = (io_header_t *)io;
00053   resource_table_t *res = (resource_table_t *)(io + h->
    resources);
00054   dword *tab = (dword *)((byte *)res + res->sound_table);
00055   sound_header_t *data;
00056   byte *temp;
00057   byte *src;
00058   byte *dst;
00059   char ofs;
00060   int count;
00061   for (int i = 0; i < res->sound_count; i++) {
00062     data = (sound_header_t *)((byte *)tab + *tab);
00063     if (data->type == SOUND_TYPE2 || data->type ==
    SOUND_TYPE1)
00064       if (data->packed == 1) {
00065     count = data->size - sizeof(sound_header_t) >> 1;
00066     temp = malloc(count);
00067     memcpy(temp, data + 1, count);
00068     src = temp;
00069     dst = (byte *)(data + 1);
00070     ofs = *(char *)src++;
00071     *dst++ = ofs;
00072     for (int j = 0; j < count - 1; j++) {
00073       ofs += sound_offset_table[*src >> 4];
00074       *dst++ = ofs;
00075       ofs += sound_offset_table[*src++ & 0xf];
00076       *dst++ = ofs;
00077     }
00078     dst[-1] = dst[-2] = 0;
00079     data->size -= 2;
00080     free(temp);
00081       }
00082     ++tab;
00083   }
00084 }
```

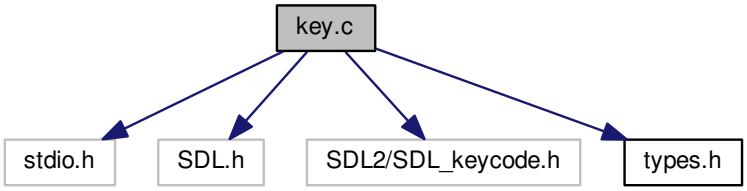
5.12 io.h

```
00001 #include "types.h"
00002
00003 #pragma pack(1)
00004
00005 typedef struct {
00006   word id;
00007   word main_entry;
00008   byte control;
00009   byte version;
00010   word key_entry;
00011   byte u1;
00012   byte u2;
00013   word message_entry;
00014   byte u4;
00015   byte u5;
00016   dword resources;
00017   word stack_size;
00018   word data_size;
00019   word msg_size;
00020 } io_header_t;
00021
00022 void io_unpack_sound(byte *io);
00023 int io_num_free_files();
```

5.13 Файл key.c

Работа с клавиатурой

#include <stdio.h>
#include <SDL.h>
#include <SDL2/SDL_keycode.h>
#include "types.h"
Граф включаемых заголовочных файлов для key.c:



Функции

- void set_key (int scan, int sym, int mod)
- void release_key (int scan)
- byte get_key ()

  возвращает символ нажатой клавишу или 0 - если не нажата
- byte get_key_mod ()

  возвращает состояние клавиш Shift, Alt, Ctrl.
- byte wait_for_key ()

Переменные

- byte key_symbol
- int key_mod = 0

5.13.1 Подробное описание

Работа с клавиатурой

Автор

alex <alex>

Дата

Sun Jan 30 16:52:27 2022

См. определение в файле key.c

5.13.2 Функции

5.13.2.1 byte get_key ( )

возвращает символ нажатой клавишу или 0 - если не нажата

См. определение в файле key.c строка

```
00071 {
00072   return key_symbol;
00073 }
```

5.13.2.2 byte get_key_mod ( )

возвращает состояние клавиш Shift, Alt, Ctrl.

< все Shift Alt Ctrl

См. определение в файле key.c строка

```
00077 {
00078   return (byte)(key_mod & 0x1f);
00079 }
```

5.13.2.3 void release_key (int scan)

Обработка события отпускания клавиши

Аргументы

scan	скан код клавиши

См. определение в файле key.c строка

```
00065 {
00066   key_symbol = 0;
00067 }
```

5.13.2.4 void set_key (int scan, int sym, int mod)

Обработка события нажатия клавиши

Аргументы

scan	скан код клавиши
mod	модификаторы Alt Ctrl Shift

См. определение в файле key.c строка

```
00026 {
00027   key_symbol = sym;
00028   key_mod = mod;
00029   if (scan >= SDL_SCANCODE_F1 && scan <= SDL_SCANCODE_F10)
00030     key_symbol = 0xbb + scan - SDL_SCANCODE_F1;
00031   else if (scan == SDL_SCANCODE_KP_8)
00032     key_symbol = '8';
00033   else if (scan == SDL_SCANCODE_KP_2)
00034     key_symbol = '2';
00035   else if (scan == SDL_SCANCODE_KP_4)
00036     key_symbol = '4';
00037   else if (scan == SDL_SCANCODE_KP_6)
00038     key_symbol = '6';
00039   else if (scan == SDL_SCANCODE_KP_7)
00040     key_symbol = '7';
00041   else if (scan == SDL_SCANCODE_KP_1)
00042     key_symbol = '1';
00043   else if (scan == SDL_SCANCODE_KP_9)
00044     key_symbol = '9';
00045   else if (scan == SDL_SCANCODE_KP_3)
00046     key_symbol = '3';
00047   else if (scan == SDL_SCANCODE_KP_0)
00048     key_symbol = '0';
00049   else if (scan == SDL_SCANCODE_ESCAPE)
00050     key_symbol = 0x1b;
00051   else if (scan == SDL_SCANCODE_BACKSPACE)
00052     key_symbol = 0x8;
00053   else if (scan == SDL_SCANCODE_TAB)
00054     key_symbol = 0x9;
00055   else if (scan == SDL_SCANCODE_RETURN)
00056     key_symbol = 0x0d;
00057 }
```

5.13.2.5 byte wait_for_key ( )

См. определение в файле key.c строка

```
00082 {
00083   SDL_Event e;
00084
00085   while (1)
00086     if (SDL_PollEvent(&e))
00087       if (e.type == SDL_KEYDOWN) {
00088     set_key(e.key.keysym.scancode, e.key.keysym.sym, e.key.keysym.mod);
00089     break;
00090       }
00091   return get_key();
00092 }
```

5.13.3 Переменные

5.13.3.1 int key_mod = 0

нажатые модификаторы

См. определение в файле key.c строка

5.13.3.2 byte key_symbol

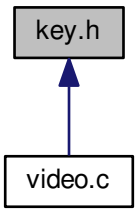
См. определение в файле key.c строка

5.14 key.c

```
#include <stdio.h>
#include <SDL.h>
#include <SDL2/SDL_keycode.h>
#include "types.h"

byte key_symbol;

int key_mod = 0;
void set_key(int scan, int sym, int mod)
{
  key_symbol = sym;
  key_mod = mod;
  if (scan >= SDL_SCANCODE_F1 && scan <= SDL_SCANCODE_F10)
    key_symbol = 0xbb + scan - SDL_SCANCODE_F1;
  else if (scan == SDL_SCANCODE_KP_8)
    key_symbol = '8';
  else if (scan == SDL_SCANCODE_KP_2)
    key_symbol = '2';
  else if (scan == SDL_SCANCODE_KP_4)
    key_symbol = '4';
  else if (scan == SDL_SCANCODE_KP_6)
    key_symbol = '6';
  else if (scan == SDL_SCANCODE_KP_7)
    key_symbol = '7';
  else if (scan == SDL_SCANCODE_KP_1)
    key_symbol = '1';
  else if (scan == SDL_SCANCODE_KP_9)
    key_symbol = '9';
  else if (scan == SDL_SCANCODE_KP_3)
    key_symbol = '3';
  else if (scan == SDL_SCANCODE_KP_0)
    key_symbol = '0';
  else if (scan == SDL_SCANCODE_ESCAPE)
    key_symbol = 0x1b;
  else if (scan == SDL_SCANCODE_BACKSPACE)
    key_symbol = 0x8;
  else if (scan == SDL_SCANCODE_TAB)
    key_symbol = 0x9;
  else if (scan == SDL_SCANCODE_RETURN)
    key_symbol = 0x0d;
}

void release_key(int scan)
{
  key_symbol = 0;
}

byte get_key()
{
  return key_symbol;
}

byte get_key_mod()
{
  return (byte)(key_mod & 0x1f);
}

byte wait_for_key()
{
  SDL_Event e;

  while (1)
    if (SDL_PollEvent(&e))
      if (e.type == SDL_KEYDOWN) {
    set_key(e.key.keysym.scancode, e.key.keysym.sym, e.key.keysym.mod);
    break;
      }
  return get_key();
}
```

5.15 Файл key.h

Граф файлов, в которые включается этот файл:



Функции

- void set_key (int scan, int sym, int mod)
- void release_key (int scan)
- byte get_key ()

  возвращает символ нажатой клавишу или 0 - если не нажата
- byte get_key_mod ()

  возвращает состояние клавиш Shift, Alt, Ctrl.
- byte wait_for_key ()

5.15.1 Функции

5.15.1.1 byte get_key ( )

возвращает символ нажатой клавишу или 0 - если не нажата

См. определение в файле key.c строка

```
00071 {
00072   return key_symbol;
00073 }
```

5.15.1.2 byte get_key_mod ( )

возвращает состояние клавиш Shift, Alt, Ctrl.

< все Shift Alt Ctrl

См. определение в файле key.c строка

```
00077 {
00078   return (byte)(key_mod & 0x1f);
00079 }
```

5.15.1.3 void release_key (int scan)

Обработка события отпускания клавиши

Аргументы

scan	скан код клавиши

См. определение в файле key.c строка

```
00065 {
00066   key_symbol = 0;
00067 }
```

5.15.1.4 void set_key (int scan, int sym, int mod)

Обработка события нажатия клавиши

Аргументы

scan	скан код клавиши
mod	модификаторы Alt Ctrl Shift

См. определение в файле key.c строка

```
00026 {
00027   key_symbol = sym;
00028   key_mod = mod;
00029   if (scan >= SDL_SCANCODE_F1 && scan <= SDL_SCANCODE_F10)
00030     key_symbol = 0xbb + scan - SDL_SCANCODE_F1;
00031   else if (scan == SDL_SCANCODE_KP_8)
00032     key_symbol = '8';
00033   else if (scan == SDL_SCANCODE_KP_2)
00034     key_symbol = '2';
00035   else if (scan == SDL_SCANCODE_KP_4)
00036     key_symbol = '4';
00037   else if (scan == SDL_SCANCODE_KP_6)
00038     key_symbol = '6';
00039   else if (scan == SDL_SCANCODE_KP_7)
00040     key_symbol = '7';
00041   else if (scan == SDL_SCANCODE_KP_1)
00042     key_symbol = '1';
00043   else if (scan == SDL_SCANCODE_KP_9)
00044     key_symbol = '9';
00045   else if (scan == SDL_SCANCODE_KP_3)
00046     key_symbol = '3';
00047   else if (scan == SDL_SCANCODE_KP_0)
00048     key_symbol = '0';
00049   else if (scan == SDL_SCANCODE_ESCAPE)
00050     key_symbol = 0x1b;
00051   else if (scan == SDL_SCANCODE_BACKSPACE)
00052     key_symbol = 0x8;
00053   else if (scan == SDL_SCANCODE_TAB)
00054     key_symbol = 0x9;
00055   else if (scan == SDL_SCANCODE_RETURN)
00056     key_symbol = 0x0d;
00057 }
```

5.15.1.5 byte wait_for_key ( )

См. определение в файле key.c строка

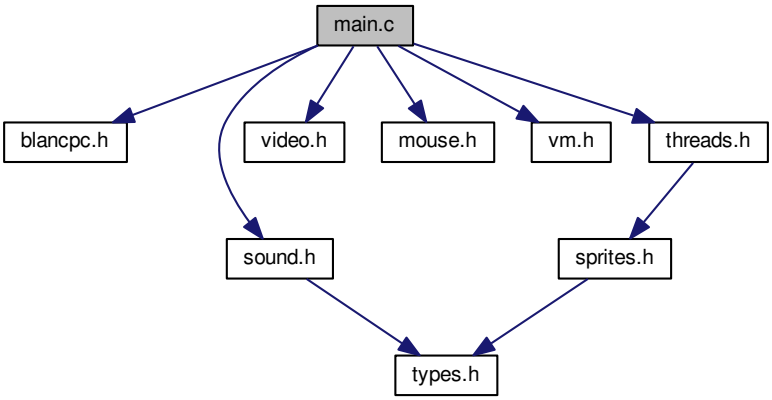
```
{
  SDL_Event e;

  while (1)
    if (SDL_PollEvent(&e))
      if (e.type == SDL_KEYDOWN) {
    set_key(e.key.keysym.scancode, e.key.keysym.sym, e.key.keysym.mod);
    break;
      }
  return get_key();
}
```

5.16 key.h

```
void set_key(int scan, int sym, int mod);
void release_key(int scan);
byte get_key();
byte get_key_mod();
byte wait_for_key();
```

5.17 Файл main.c

#include "blancpc.h"
#include "sound.h"
#include "video.h"
#include "mouse.h"
#include "vm.h"
#include "threads.h"
Граф включаемых заголовочных файлов для main.c:



Функции

- void global_init ()
- void main ()

5.17.1 Функции

5.17.1.1 void global_init ( )

Инициализация всех подсистем

См. определение в файле main.c строка

```
00012 {
00013   load_blancpc();
00014   video_init();
00015   sound_init();
00016   mouse_init();
00017 }
```

5.17.1.2 void main ( )

См. определение в файле main.c строка

```
00020 {
00021   global_init();
00022   threads_init();
00023   vm_init();
00024   while (1) {
00025     threads_run();
00026     video_update();
00027   }
00028 }
```

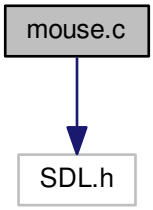
5.18 main.c

```
00001 #include "blancpc.h"
00002 #include "sound.h"
00003 #include "video.h"
00004 #include "mouse.h"
00005 #include "vm.h"
00006 #include "threads.h"
00007
00011 void global_init()
00012 {
00013   load_blancpc();
00014   video_init();
00015   sound_init();
00016   mouse_init();
00017 }
00018
00019 void main()
00020 {
00021   global_init();
00022   threads_init();
00023   vm_init();
00024   while (1) {
00025     threads_run();
00026     video_update();
00027   }
00028 }
```

5.19 Файл mouse.c

Управление курсором мыши

#include <SDL.h>
Граф включаемых заголовочных файлов для mouse.c:



Функции

- void mouse_event (int x, int y, int buttons)
- void mouse_get (int *x, int *y, int *buttons)
- void mouse_init ()
- void mouse_draw_cursor ()

Переменные

- int show_cursor = -1
- int mouse_x
- int mouse_y
- int mouse_buttons

5.19.1 Подробное описание

Управление курсором мыши

Автор

alex <alex>

Дата

Mon Apr 4 08:05:40 2022

Системный курсор скрыт, вместо него рисуется изображение курсора

См. определение в файле mouse.c

5.19.2 Функции

5.19.2.1 void mouse_draw_cursor ( )

См. определение в файле mouse.c строка

```
00048 {
00049 }
```

5.19.2.2 void mouse_event (int x, int y, int buttons)

Установка параметров при событиях мыши Может использоваться при установке курсора мыши в новую позицию

Аргументы

x	координата x
y	координата y
buttons	кнопки

См. определение в файле mouse.c строка

```
00026 {
00027   mouse_x = x;
00028   mouse_y = y;
00029   mouse_buttons = buttons;
00030 }
```

5.19.2.3 void mouse_get (int * x, int * y, int * buttons)

Получение состояния мыши

См. определение в файле mouse.c строка

```
00036 {
00037   *x = mouse_x;
00038   *y = mouse_y;
00039   *buttons = mouse_buttons;
00040 }
```

5.19.2.4 void mouse_init ( )

См. определение в файле mouse.c строка

```
00043 {
00044   SDL_ShowCursor(SDL_FALSE);
00045 }
```

5.19.3 Переменные

5.19.3.1 int mouse_buttons

состояние кнопок мыши

См. определение в файле mouse.c строка

5.19.3.2 int mouse_x

координаты курсора

См. определение в файле mouse.c строка

5.19.3.3 int mouse_y

См. определение в файле mouse.c строка

5.19.3.4 int show_cursor = -1

флаг курсора мыши

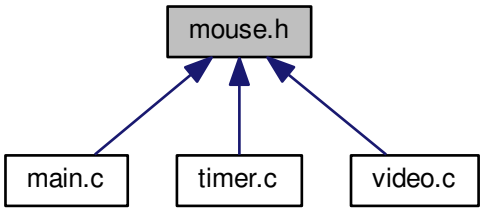
См. определение в файле mouse.c строка

5.20 mouse.c

```
00001
00010 #include <SDL.h>
00011
00012 int show_cursor = -1;
00013 int mouse_x;
00014 int mouse_y;
00015 int mouse_buttons;
00025 void mouse_event(int x, int y, int buttons)
00026 {
00027   mouse_x = x;
00028   mouse_y = y;
00029   mouse_buttons = buttons;
00030 }
00031
00035 void mouse_get(int *x, int *y, int *buttons)
00036 {
00037   *x = mouse_x;
00038   *y = mouse_y;
00039   *buttons = mouse_buttons;
00040 }
00041
00042 void mouse_init()
00043 {
00044   SDL_ShowCursor(SDL_FALSE);
00045 }
00046
00047 void mouse_draw_cursor()
00048 {
00049 }
```

5.21 Файл mouse.h

Граф файлов, в которые включается этот файл:



Функции

- void mouse_init ()
- void mouse_draw_cursor ()
- void mouse_event (int x, int y, int buttons)
- void mouse_get (int *x, int *y, int *buttons)

5.21.1 Функции

5.21.1.1 void mouse_draw_cursor ( )

См. определение в файле mouse.c строка

```
00048 {
00049 }
```

5.21.1.2 void mouse_event (int x, int y, int buttons)

Установка параметров при событиях мыши Может использоваться при установке курсора мыши в новую позицию

Аргументы

x	координата x
y	координата y
buttons	кнопки

См. определение в файле mouse.c строка

```
00026 {
```

```
00027   mouse_x = x;
00028   mouse_y = y;
00029   mouse_buttons = buttons;
00030 }
```

5.21.1.3 void mouse_get (int * x, int * y, int * buttons)

Получение состояния мыши

См. определение в файле mouse.c строка

```
00036 {
00037   *x = mouse_x;
00038   *y = mouse_y;
00039   *buttons = mouse_buttons;
00040 }
```

5.21.1.4 void mouse_init ( )

См. определение в файле mouse.c строка

```
00043 {
00044   SDL_ShowCursor(SDL_FALSE);
00045 }
```

5.22 mouse.h

```
00001 void mouse_init();
00002 void mouse_draw_cursor();
00003 void mouse_event(int x, int y, int buttons);
00004 void mouse_get(int *x, int *y, int *buttons);
```

5.23 Файл palette.c

Функции

- void palette_update ()

5.23.1 Функции

5.23.1.1 void palette_update ( )

См. определение в файле palette.c строка

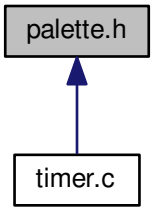
```
00002 {
00003 }
```

5.24 palette.c

```
00001 void palette_update()
00002 {
00003 }
```

5.25 Файл palette.h

Граф файлов, в которые включается этот файл:



Функции

- void palette_update ()

5.25.1 Функции

5.25.1.1 void palette_update ( )

См. определение в файле palette.c строка

```
00002 {
00003 }
```

5.26 palette.h

```
00001 void palette_update();
```

5.27 Файл README.md

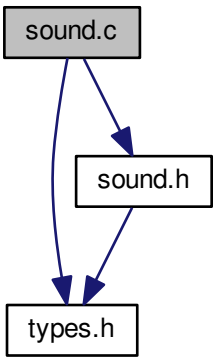
5.28 README.md

```
00001 # alis-c
```

5.29 Файл sound.c

Модуль обработки звуков (синтезатор + оцифрованный звук)

#include "types.h"
#include "sound.h"
Граф включаемых заголовочных файлов для sound.c:



Макросы

- #define NUM_CHANNELS 4
- #define MAX_VOLUME 32767
- #define CHANNEL_OFF (1 << 7)

Функции

- void sound_find_channel_max_priority ()
- void synth_play (sound_channel_t *ch)
- void sound_channel_off (sound_channel_t *ch)
- void sound_process_channel (sound_channel_t *ch)
- void sound_update ()
- void sound_init ()

Переменные

- sound_channel_t sound_channels [NUM_CHANNELS]
- sound_channel_t * current_channel
- int channel_num
- int channel_num_max_priority

5.29.1 Подробное описание

Модуль обработки звуков (синтезатор + оцифрованный звук)

Автор

alex <alex>

Дата

Tue Apr 5 07:29:09 2022

Первые 3 канала могут использоваться как синтезатор Все 4 канала могут использоваться под оцифрованный звук

См. определение в файле sound.c

5.29.2 Макросы

5.29.2.1 #define CHANNEL_OFF (1 << 7)

флаг - канал отключен

См. определение в файле sound.c строка

5.29.2.2 #define MAX_VOLUME 32767

порог громкости

См. определение в файле sound.c строка

5.29.2.3 #define NUM_CHANNELS 4

число каналов звука

См. определение в файле sound.c строка

5.29.3 Функции

5.29.3.1 void sound_channel_off (sound_channel_t * ch)

Выключение канала звука

Аргументы

ch	канал

См. определение в файле sound.c строка

```
00047 {
00048   ch->volume = 0;
00049   ch->frequency = 0;
00050   ch->priority = -128;
00051   ch->synth = 0;
00052   ch->flags = 0;
00053   synth_play(ch);
00054 }
```

5.29.3.2 void sound_find_channel_max_priority ( )

Вычисляется номер канала, у которого максимальный приоритет

См. определение в файле sound.c строка

```
00029 {
00030 }
```

5.29.3.3 void sound_init ( )

Инициализация звуковой системы Инициализация каналов

См. определение в файле sound.c строка

```
00111 {
00112 }
```

5.29.3.4 void sound_process_channel (sound_channel_t * ch)

Обработка канала звука

Звук звучит с определенной длительностью Громкость может изменяться (усиление/затухание) звука

Аргументы

ch	текущий канал звука

См. определение в файле sound.c строка

```
00064 {
00065   short vol;
00066   current_channel = ch;
00067   channel_num = ch - sound_channels;
00068   if (ch->synth < 0)
00069     return;
00070   --ch->length;
00071   if (!ch->length) {
00072     sound_channel_off(ch);
00073     return;
00074   }
```

```
vol = ch->volume + ch->gain;
if (vol < 0) {
  sound_channel_off(ch);
  return;
}
if (vol > MAX_VOLUME)
  vol = MAX_VOLUME;
ch->volume = vol;
vol = ch->frequency + ch->frequency_gain;
if (vol < 0)
  sound_channel_off(ch);
else {
  ch->frequency = vol;
  synth_play(ch);
}
}
```

5.29.3.5 void sound_update ( )

Обновление звуков. Проверка 3-х каналов, что там появились звуки

См. определение в файле sound.c строка

```
{
sound_channel_t *ch = sound_channels;
sound_find_channel_max_priority();
for (int i = 0; i < NUM_CHANNELS - 1; i++, ch++)
  if (ch->flags > 0)
    if (!(ch->flags & CHANNEL_OFF))
  sound_process_channel(ch);
}
```

5.29.3.6 void synth_play (sound_channel_t * ch)

Проиграть ноту на синтезаторе

Аргументы

ch	в канале - параметры ноты

См. определение в файле sound.c строка

```
{
}
```

5.29.4 Переменные

5.29.4.1 int channel_num

номер текущего канала

См. определение в файле sound.c строка

5.29.4.2 int channel_num_max_priority

См. определение в файле sound.c строка

5.29.4.3 sound_channel_t* current_channel

текущий канал

См. определение в файле sound.c строка

5.29.4.4 sound_channel_t sound_channels[NUM_CHANNELS]

каналы звука

См. определение в файле sound.c строка

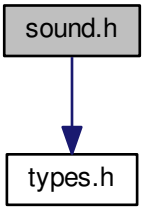
5.30 sound.c

```
00001
00012 #include "types.h"
00013 #include "sound.h"
00014
00015 #define NUM_CHANNELS 4
00016 #define MAX_VOLUME 32767
00018 #define CHANNEL_OFF (1 << 7)
00020 sound_channel_t  sound_channels[NUM_CHANNELS];
00021 sound_channel_t *current_channel;
00022 int channel_num;
00023 int channel_num_max_priority;
00024
00028 void sound_find_channel_max_priority()
00029 {
00030 }
00031
00037 void synth_play(sound_channel_t *ch)
00038 {
00039 }
00040
00046 void sound_channel_off(sound_channel_t *ch)
00047 {
00048   ch->volume = 0;
00049   ch->frequency = 0;
00050   ch->priority = -128;
00051   ch->synth = 0;
00052   ch->flags = 0;
00053   synth_play(ch);
00054 }
00055
00063 void sound_process_channel(sound_channel_t *ch)
00064 {
00065   short vol;
00066   current_channel = ch;
00067   channel_num = ch - sound_channels;
00068   if (ch->synth < 0)
00069     return;
00070   --ch->length;
00071   if (!ch->length) {
00072     sound_channel_off(ch);
00073     return;
00074   }
00075   vol = ch->volume + ch->gain;
00076   if (vol < 0) {
00077     sound_channel_off(ch);
00078     return;
00079   }
00080   if (vol > MAX_VOLUME)
00081     vol = MAX_VOLUME;
00082   ch->volume = vol;
00083   vol = ch->frequency + ch->frequency_gain;
00084   if (vol < 0)
00085     sound_channel_off(ch);
00086   else {
00087     ch->frequency = vol;
00088     synth_play(ch);
00089   }
00090 }
00091
00096 void sound_update()
```

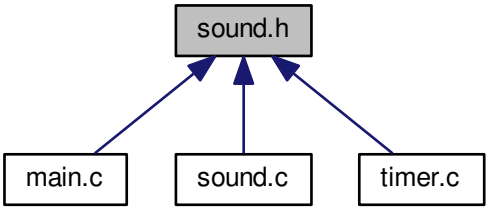
```
{
  sound_channel_t *ch = sound_channels;
  sound_find_channel_max_priority();
  for (int i = 0; i < NUM_CHANNELS - 1; i++, ch++)
    if (ch->flags > 0)
      if (!(ch->flags & CHANNEL_OFF))
    sound_process_channel(ch);
}

void sound_init()
{
}
```

5.31 Файл sound.h

#include "types.h"
Граф включаемых заголовочных файлов для sound.h:



Граф файлов, в которые включается этот файл:



Структуры данных

- struct sound_channel_t

Функции

- void sound_init ()
- void sound_update ()

5.31.1 Функции

5.31.1.1 void sound_init ( )

Инициализация звуковой системы Инициализация каналов

См. определение в файле sound.c строка

```
00111 {
00112 }
```

5.31.1.2 void sound_update ( )

Обновление звуков. Проверка 3-х каналов, что там появились звуки

См. определение в файле sound.c строка

```
00097 {
00098   sound_channel_t *ch = sound_channels;
00099   sound_find_channel_max_priority();
00100   for (int i = 0; i < NUM_CHANNELS - 1; i++, ch++)
00101     if (ch->flags > 0)
00102       if (!(ch->flags & CHANNEL_OFF))
00103     sound_process_channel(ch);
00104 }
```

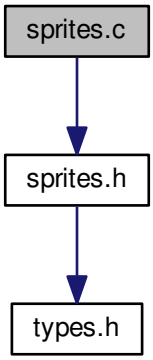
5.32 sound.h

```
00001 #include "types.h"
00002
00003 typedef struct {
00004   byte flags;
00005   char priority;
00006   char synth;
00007   word length;
00008   short volume;
00009   short gain;
00010   short frequency;
00011   short frequency_gain;
00012 } sound_channel_t;
00013
00014 void sound_init();
00015 void sound_update();
```

5.33 Файл sprites.c

Модуль для работы со спрайтами

#include "sprites.h"
Граф включаемых заголовочных файлов для sprites.c:



Функции

- int sprites_free_num ()
- void sprites_set_translate (word *vec)
- void sprites_translate (sprite_t *list, word *vec)

Переменные

- sprite_t * free_sprites

5.33.1 Подробное описание

Модуль для работы со спрайтами

Автор

alex <alex>

Дата

Fri Apr 8 16:00:53 2022

См. определение в файле sprites.c

5.33.2 Функции

5.33.2.1 int sprites_free_num ( )

Возвращает свободное число ячеек в таблице спрайтов

См. определение в файле sprites.c строка

```
00017 {
00018   int num = 0;
00019   sprite_t *s = free_sprites;
00020   while (s) {
00021     num++;
00022     s = s->next;
00023   }
00024   return num;
00025 }
```

5.33.2.2 void sprites_set_translate (word * vec)

См. определение в файле sprites.c строка

```
00028 {
00029 }
```

5.33.2.3 void sprites_translate (sprite_t * list, word * vec)

См. определение в файле sprites.c строка

```
00032 {
00033 }
```

5.33.3 Переменные

5.33.3.1 sprite_t* free_sprites

голова списка своболных спрайтов

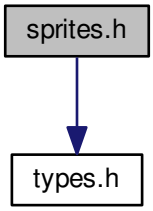
См. определение в файле sprites.c строка

5.34 sprites.c

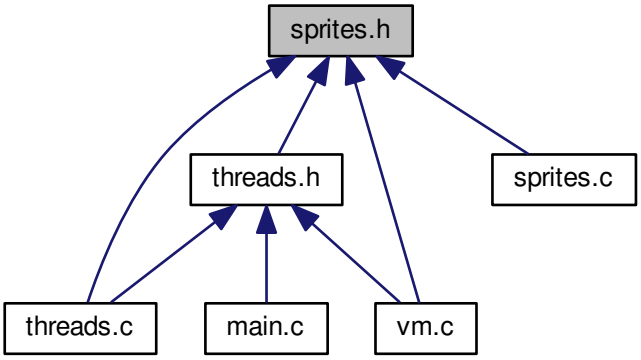
```
00001
00009 #include "sprites.h"
00010
00011 sprite_t *free_sprites;
00016 int sprites_free_num()
00017 {
00018   int num = 0;
00019   sprite_t *s = free_sprites;
00020   while (s) {
00021     num++;
00022     s = s->next;
00023   }
00024   return num;
00025 }
00026
00027 void sprites_set_translate(word *vec)
00028 {
00029 }
00030
00031 void sprites_translate(sprite_t *list, word *vec)
00032 {
00033 }
```

5.35 Файл sprites.h

#include "types.h"
Граф включаемых заголовочных файлов для sprites.h:



Граф файлов, в которые включается этот файл:



Структуры данных

- struct sprite_t

Функции

- int sprites_free_num ()
- void sprites_set_translate (word *vec)
- void sprites_translate (sprite_t *list, word *vec)

5.35.1 Функции

5.35.1.1 int sprites_free_num ( )

Возвращает свободное число ячеек в таблице спрайтов

См. определение в файле sprites.c строка

```
00017 {
00018   int num = 0;
00019   sprite_t *s = free_sprites;
00020   while (s) {
00021     num++;
00022     s = s->next;
00023   }
00024   return num;
00025 }
```

5.35.1.2 void sprites_set_translate (word * vec)

См. определение в файле sprites.c строка

```
00028 {
00029 }
```

5.35.1.3 void sprites_translate (sprite_t * list, word * vec)

См. определение в файле sprites.c строка

```
00032 {
00033 }
```

5.36 sprites.h

```
00001 #ifndef SPRITES
00002 #define SPRITES
00003
00004 #include "types.h"
00005
00006 typedef struct sprite_s {
00007   struct sprite_s *next;
00008 } sprite_t;
00009
00010 int sprites_free_num();
00011 void sprites_set_translate(word *vec);
00012 void sprites_translate(sprite_t *list, word *vec);
00013
00014 #endif
```

5.37 Файл text.c

Функции

- void set_cursor_pos (int x, int y)

Переменные

- int cursor_x
- int cursor_y

5.37.1 Функции

5.37.1.1 void set_cursor_pos (int x, int y)

См. определение в файле text.c строка

```
00005 {
00006   cursor_x = x;
00007   cursor_y = y;
00008 }
```

5.37.2 Переменные

5.37.2.1 int cursor_x

позиция курсора для вывода текста

См. определение в файле text.c строка

5.37.2.2 int cursor_y

См. определение в файле text.c строка

5.38 text.c

```
00001 int cursor_x;
00002 int cursor_y;
00003
00004 void set_cursor_pos(int x, int y)
00005 {
00006   cursor_x = x;
00007   cursor_y = y;
00008 }
```

5.39 Файл text.h

Функции

- void set_cursor_pos (int x, int y)

5.39.1 Функции

5.39.1.1 void set_cursor_pos (int x, int y)

См. определение в файле text.c строка

```
00005 {
00006  cursor_x = x;
00007  cursor_y = y;
00008 }
```

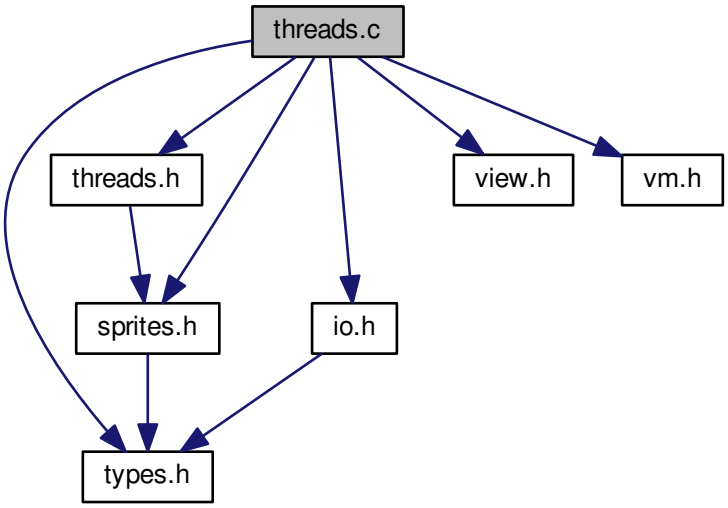
5.40 text.h

```
00001 void set_cursor_pos(int x, int y);
```

5.41 Файл threads.c

Модуль работы с потоками

#include "types.h"
#include "threads.h"
#include "io.h"
#include "sprites.h"
#include "view.h"
#include "vm.h"
Граф включаемых заголовочных файлов для threads.c:



Функции

- void threads_init ()
- void threads_run ()
- int threads_free_num ()

Переменные

- word * threads_list_pos
- word threads_list [256]
- int max_threads
- int loaded_threads
- threads_table_t * threads_table
- thread_t * current_thread
- int thread_flag
- int no_save

5.41.1 Подробное описание

Модуль работы с потоками

Автор

alex <alex>

Дата

Thu Apr 7 10:23:17 2022

См. определение в файле threads.c

5.41.2 Функции

5.41.2.1 int threads_free_num ( )

Возвращает свободное число ячеек в таблице потоков

См. определение в файле threads.c строка

```
00083 {
00084   return max_threads - loaded_threads;
00085 }
```

5.41.2.2 void threads_init ( )

См. определение в файле threads.c строка

```
00026 {
00027   threads_list_pos = threads_list;
00028 }
```

5.41.2.3 void threads_run ( )

Запуск потоков Цикл по всем потокам из таблицы потоков Запускается обработка сообщения, если есть сообщения, они не запрещены и у потока есть обработчик Основная программа (вместе с обработчиком клавиш) потока не запускается, если он на паузе или не на своем кадре В основной программе меняются ip и указатель стека, обработка сообщений и клавиш начинается всегда с одного места После запуска виртуальной машины происходит перемещение спрайтов относительно нового начала координат После того как все потоки отработали происходит обновление экрана

См. определение в файле threads.c строка

```
00040 {
00041   threads_table_t *tt;
00042   int ip;
00043   word sp;
00044   io_header_t *h;
00045   for (tt = threads_table; tt; tt = tt->next) {
00046     current_thread = tt->thread;
00047     thread_flag = 0;
00048     no_save = 0;
00049     h = (io_header_t *)current_thread->io_file;
00050     if (current_thread->flags & THREAD_MSG && !(
    current_thread->flags & THREAD_NOSTART3) && h->
    message_entry) {
00051       sp = current_thread->call_sp;
00052       sprites_set_translate((word *)current_thread->
    memory);
00053       vm_run(current_thread->io_file + h->
    message_entry + 0xa);
00054       current_thread->call_sp = sp;
00055       sprites_translate(current_thread->sprites, (
    word *)current_thread->memory);
00056     }
00057     if (!current_thread->running)
00058       continue;
00059     if (current_thread->running < 0)
00060       current_thread->running = 1;
00061     --current_thread->frame_num;
00062     if (current_thread->frame_num)
00063       continue;
00064     sprites_set_translate((word *)current_thread->
    memory);
00065     no_save++;
00066     current_thread->ip = vm_run(current_thread->
    io_file + current_thread->ip) - current_thread->
    io_file;
00067     no_save = 0;
00068     if (h->key_entry) {
00069       sp = current_thread->call_sp;
00070       vm_run(current_thread->io_file + h->key_entry);
00071       current_thread->call_sp = sp;
00072     }
00073     sprites_translate(current_thread->sprites, (
    word *)current_thread->memory);
00074     current_thread->frame_num = current_thread->
    num_frames;
00075   }
00076   view_update();
00077 }
```

5.41.3 Переменные

5.41.3.1 thread_t* current_thread

текущий выполняемый поток

См. определение в файле threads.c строка

5.41.3.2 int loaded_threads

число загруженных потоков

См. определение в файле threads.c строка

5.41.3.3 int max_threads

максимальное число потоков

См. определение в файле threads.c строка

5.41.3.4 int no_save

См. определение в файле threads.c строка

5.41.3.5 int thread_flag

См. определение в файле threads.c строка

5.41.3.6 word threads_list[256]

список номеров потоков

См. определение в файле threads.c строка

5.41.3.7 word* threads_list_pos

указатель на массив номеров потоков

См. определение в файле threads.c строка

5.41.3.8 threads_table_t* threads_table

таблица потоков

См. определение в файле threads.c строка

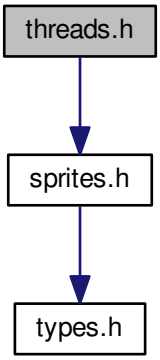
5.42 threads.c

```
#include "types.h"
#include "threads.h"
#include "io.h"
#include "sprites.h"
#include "view.h"
#include "vm.h"

word *threads_list_pos;
word threads_list[256];
int max_threads;
int loaded_threads;
threads_table_t *threads_table;
thread_t *current_thread;
int thread_flag;
int no_save;

void threads_init()
{
  threads_list_pos = threads_list;
}

void threads_run()
{
  threads_table_t *tt;
  int ip;
  word sp;
  io_header_t *h;
  for (tt = threads_table; tt; tt = tt->next) {
    current_thread = tt->thread;
    thread_flag = 0;
    no_save = 0;
    h = (io_header_t *)current_thread->io_file;
    if (current_thread->flags & THREAD_MSG && !(current_thread->flags & THREAD_NOSTART3) && h->message_entry) {
      sp = current_thread->call_sp;
      sprites_set_translate((word *)current_thread->memory);
      vm_run(current_thread->io_file + h->message_entry + 0xa);
      current_thread->call_sp = sp;
      sprites_translate(current_thread->sprites, (word *)current_thread->memory);
    }
    if (!current_thread->running)
      continue;
    if (current_thread->running < 0)
      current_thread->running = 1;
    --current_thread->frame_num;
    if (current_thread->frame_num)
      continue;
    sprites_set_translate((word *)current_thread->memory);
    no_save++;
    current_thread->ip = vm_run(current_thread->io_file + current_thread->ip) - current_thread->io_file;
    no_save = 0;
    if (h->key_entry) {
      sp = current_thread->call_sp;
      vm_run(current_thread->io_file + h->key_entry);
      current_thread->call_sp = sp;
    }
    sprites_translate(current_thread->sprites, (word *)current_thread->memory);
    current_thread->frame_num = current_thread->num_frames;
  }
  view_update();
}

int threads_free_num()
{
  return max_threads - loaded_threads;
}
```

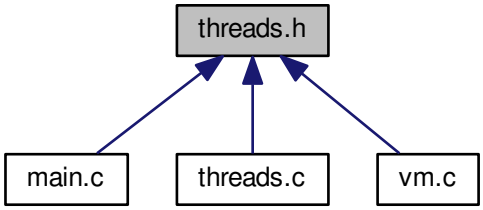
5.43 Файл threads.h

#include "sprites.h"

Граф включаемых заголовочных файлов для threads.h:



Граф файлов, в которые включается этот файл:



Структуры данных

- struct thread_t

  структура потока
- struct threads_table_t

  запись таблицы потоков

Макросы

- #define THREAD_NOMSG (1 << 0)
- #define THREAD_NOSTART3 (1 << 1)
- #define THREAD_MSG (1 << 7)

Функции

- void threads_init ()
- void threads_run ()
- int threads_free_num ()

5.43.1 Макросы

5.43.1.1 #define THREAD_MSG (1 << 7)

есть сообщения для потока

См. определение в файле threads.h строка

5.43.1.2 #define THREAD_NOMSG (1 << 0)

поток не принимает сообщения

См. определение в файле threads.h строка

5.43.1.3 #define THREAD_NOSTART3 (1 << 1)

не запуск сценария 3 каждый фрейм

См. определение в файле threads.h строка

5.43.2 Функции

5.43.2.1 int threads_free_num ( )

Возвращает свободное число ячеек в таблице потоков

См. определение в файле threads.c строка

```
00083 {
00084   return max_threads - loaded_threads;
00085 }
```

5.43.2.2 void threads_init ( )

См. определение в файле threads.c строка

```
00026 {
00027   threads_list_pos = threads_list;
00028 }
```

5.43.2.3 void threads_run ( )

Запуск потоков Цикл по всем потокам из таблицы потоков Запускается обработка сообщения, если есть сообщения, они не запрещены и у потока есть обработчик Основная программа (вместе с обработчиком клавиш) потока не запускается, если он на паузе или не на своем кадре В основной программе меняются ip и указатель стека, обработка сообщений и клавиш начинается всегда с одного места После запуска виртуальной машины происходит перемещение спрайтов относительно нового начала координат После того как все потоки отработали происходит обновление экрана

См. определение в файле threads.c строка

```
00040 {
00041   threads_table_t *tt;
00042   int ip;
00043   word sp;
00044   io_header_t *h;
00045   for (tt = threads_table; tt; tt = tt->next) {
00046     current_thread = tt->thread;
00047     thread_flag = 0;
00048     no_save = 0;
00049     h = (io_header_t *)current_thread->io_file;
00050     if (current_thread->flags & THREAD_MSG && !(
    current_thread->flags & THREAD_NOSTART3) && h->
    message_entry) {
00051       sp = current_thread->call_sp;
00052       sprites_set_translate((word *)current_thread->
    memory);
00053       vm_run(current_thread->io_file + h->
    message_entry + 0xa);
00054       current_thread->call_sp = sp;
00055       sprites_translate(current_thread->sprites, (
    word *)current_thread->memory);
00056     }
00057     if (!current_thread->running)
00058       continue;
00059     if (current_thread->running < 0)
00060       current_thread->running = 1;
00061     --current_thread->frame_num;
00062     if (current_thread->frame_num)
00063       continue;
00064     sprites_set_translate((word *)current_thread->
    memory);
00065     no_save++;
00066     current_thread->ip = vm_run(current_thread->
    io_file + current_thread->ip) - current_thread->
    io_file;
00067     no_save = 0;
00068     if (h->key_entry) {
00069       sp = current_thread->call_sp;
00070       vm_run(current_thread->io_file + h->key_entry);
00071       current_thread->call_sp = sp;
00072     }
00073     sprites_translate(current_thread->sprites, (
    word *)current_thread->memory);
00074     current_thread->frame_num = current_thread->
    num_frames;
00075   }
00076   view_update();
00077 }
```

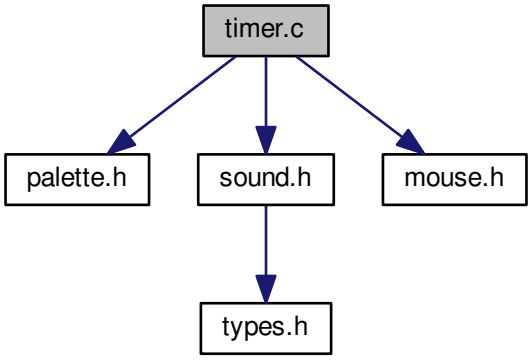
5.44 threads.h

```
00001 #include "sprites.h"
00002
00003 #define THREAD_NOMSG (1 << 0)
00004 #define THREAD_NOSTART3 (1 << 1)
00005 #define THREAD_MSG (1 << 7)
00007 typedef struct {
00009   byte flags;
00010   char running;
00011   int num_frames;
00012   int frame_num;
00013   byte *io_file;
00014   byte *memory;
00015   word ip;
```

```
00016   word call_sp;
00017   sprite_t *sprites;
00018 } thread_t;
00019
00021 typedef struct threads_table_s {
00022   thread_t *thread;
00023   struct threads_table_s *next;
00024 } threads_table_t;
00025
00026 void threads_init();
00027 void threads_run();
00028 int threads_free_num();
```

5.45 Файл timer.c

#include "palette.h"
#include "sound.h"
#include "mouse.h"
Граф включаемых заголовочных файлов для timer.c:



Функции

- void timer_update ()

Переменные

- char frame_num = 0

5.45.1 Функции

5.45.1.1 void timer_update ( )

Вызывается каждый кадр. Обновляется палитра, звук, перерисовка курсора мыши.

См. определение в файле timer.c строка

```
00012 {
00013   ++frame_num;
00014   if (!frame_num)
00015     --frame_num;
00016   palette_update();
00017   sound_update();
00018   mouse_draw_cursor();
00019 }
```

5.45.2 Переменные

5.45.2.1 char frame_num = 0

номер кадра

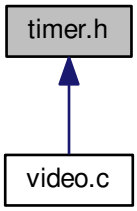
См. определение в файле timer.c строка

5.46 timer.c

```
00001 #include "palette.h"
00002 #include "sound.h"
00003 #include "mouse.h"
00004
00005 char frame_num = 0;
00011 void timer_update()
00012 {
00013   ++frame_num;
00014   if (!frame_num)
00015     --frame_num;
00016   palette_update();
00017   sound_update();
00018   mouse_draw_cursor();
00019 }
```

5.47 Файл timer.h

Граф файлов, в которые включается этот файл:



Функции

- void timer_update ()

5.47.1 Функции

5.47.1.1 void timer_update ( )

Вызывается каждый кадр. Обновляется палитра, звук, перерисовка курсора мыши.

См. определение в файле timer.c строка

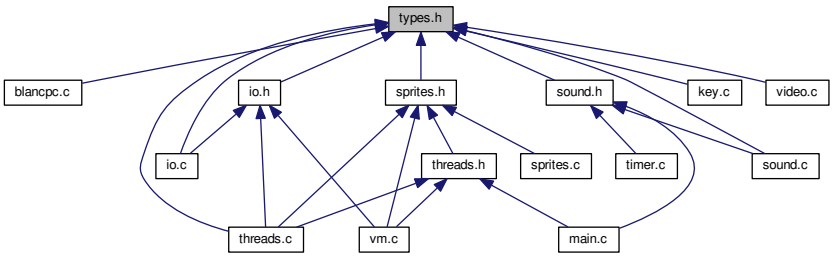
```
00012 {
00013   ++frame_num;
00014   if (!frame_num)
00015     --frame_num;
00016   palette_update();
00017   sound_update();
00018   mouse_draw_cursor();
00019 }
```

5.48 timer.h

```
00001 void timer_update();
```

5.49 Файл types.h

Граф файлов, в которые включается этот файл:



Определения типов

- typedef unsigned char byte
- typedef unsigned short word
- typedef unsigned int dword
- typedef int bool

5.49.1 Типы

5.49.1.1 typedef int bool

См. определение в файле types.h строка

5.49.1.2 typedef unsigned char byte

См. определение в файле types.h строка

5.49.1.3 typedef unsigned int dword

См. определение в файле types.h строка

5.49.1.4 typedef unsigned short word

См. определение в файле types.h строка

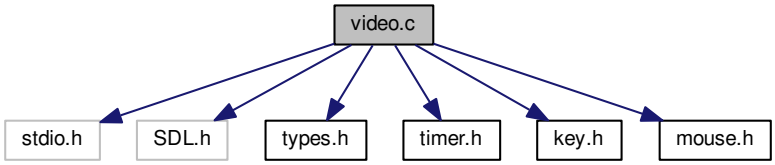
5.50 types.h

```
00001 typedef unsigned char byte;
00002 typedef unsigned short word;
00003 typedef unsigned int dword;
00004 typedef int bool;
```

5.51 Файл video.c

Модуль для работы с графикой SDL: окно, обработка событий, вывод графики

#include <stdio.h>
#include <SDL.h>
#include "types.h"
#include "timer.h"
#include "key.h"
#include "mouse.h"
Граф включаемых заголовочных файлов для video.c:



Макросы

- #define WINDOW_WIDTH 640
- #define WINDOW_HEIGHT 400
- #define SCREEN_WIDTH 320
- #define SCREEN_HEIGHT 200
- #define FPS 40

Функции

- void video_init ()
- void video_update ()
- void video_close ()
- void video_update_events ()

Переменные

- SDL_Window * window
- SDL_Renderer * renderer
- SDL_Surface * screen
- long current_time
- byte * video_buffer
- long time_step = 1000 / FPS

5.51.1 Подробное описание

Модуль для работы с графикой SDL: окно, обработка событий, вывод графики

Автор

alex <alex>

Дата

Wed Apr 6 10:35:00 2022

См. определение в файле video.c

5.51.2 Макросы

5.51.2.1 #define FPS 40

число кадров в секунду

См. определение в файле video.c строка

5.51.2.2 #define SCREEN_HEIGHT 200

См. определение в файле video.c строка

5.51.2.3 #define SCREEN_WIDTH 320

размеры буфера экрана

См. определение в файле video.c строка

5.51.2.4 #define WINDOW_HEIGHT 400

См. определение в файле video.c строка

5.51.2.5 #define WINDOW_WIDTH 640

размеры окна

См. определение в файле video.c строка

5.51.3 Функции

5.51.3.1 void video_close ( )

Завершение работы видео подсистемы

См. определение в файле video.c строка

```
{
  SDL_DestroyRenderer(renderer);
  SDL_DestroyWindow(window);
  SDL_Quit();
}
```

5.51.3.2 void video_init ( )

Инициализация видео подсистемы. Создается окно и буфер экрана

См. определение в файле video.c строка

```
{
  SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO);
  window = SDL_CreateWindow("alis", SDL_WINDOWPOS_UNDEFINED, SDL_WINDOWPOS_UNDEFINED,
    WINDOW_WIDTH, WINDOW_HEIGHT, SDL_WINDOW_SHOWN);
  if (window == NULL) {
    printf("Cannot create window\n");
    exit(1);
  }
  renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED |
    SDL_RENDERER_PRESENTVSYNC);
  if (renderer == NULL) {
    printf("Cannot create renderer\n");
    exit(1);
  }
  current_time = SDL_GetTicks();
  screen = SDL_CreateRGBSurface(0, SCREEN_WIDTH, SCREEN_HEIGHT, 8, 0, 0, 0,
    0);
  SDL_SetHint(SDL_HINT_RENDER_SCALE_QUALITY, "1");
  SDL_LockSurface(screen);
  video_buffer = screen->pixels;
}
```

5.51.3.3 void video_update ( )

Обновление экрана. Делается задержка, чтобы соблюдать частоту кадров обновляется таймер, видео буфер выводится на экран.

См. определение в файле video.c строка

```
{
  long now = SDL_GetTicks();
  if (now - current_time < time_step)
    SDL_Delay(now - current_time);
  else
    current_time = now;
  timer_update();
  SDL_UnlockSurface(screen);
  SDL_RenderClear(renderer);
  SDL_Texture* t = SDL_CreateTextureFromSurface(renderer, screen);
  SDL_RenderCopy(renderer, t, NULL, NULL);
  SDL_DestroyTexture(t);
  SDL_RenderPresent(renderer);
  SDL_LockSurface(screen);
  video_buffer = screen->pixels;
}
```

5.51.3.4 void video_update_events ( )

Обработка событий клавиатуры и мыши

См. определение в файле video.c строка

```
00091 {
00092   SDL_Event e;
00093   int x, y, b;
00094
00095   if (SDL_PollEvent(&e)) {
00096     if (e.type == SDL_QUIT) {
00097       video_close();
00098       exit(0);
00099     } else if (e.type == SDL_KEYDOWN)
00100       set_key(e.key.keysym.scancode, e.key.keysym.sym, e.key.keysym.mod);
00101     else if (e.type == SDL_KEYUP)
00102       release_key(e.key.keysym.scancode);
00103     else if (e.type == SDL_MOUSEMOTION) {
00104       b = SDL_GetMouseState(&x, &y);
00105       mouse_event(x >> 1, y >> 1, b);
00106     } else if (e.type == SDL_MOUSEBUTTONDOWN) {
00107       SDL_GetMouseState(&x, &y);
00108       mouse_event(x >> 1, y >> 1, e.button.button);
00109     } else if (e.type == SDL_MOUSEBUTTONUP) {
00110       SDL_GetMouseState(&x, &y);
00111       mouse_event(x >> 1, y >> 1, 0);
00112     }
00113   }
00114 }
```

5.51.4 Переменные

5.51.4.1 long current_time

текущее время

См. определение в файле video.c строка

5.51.4.2 SDL_Renderer* renderer

устройство вывода

См. определение в файле video.c строка

5.51.4.3 SDL_Surface* screen

поверхность экрана

См. определение в файле video.c строка

5.51.4.4 long time_step = 1000 / FPS

длительность кадра

См. определение в файле video.c строка

5.51.4.5 byte* video_buffer

видео буфер экрана 320 на 200

См. определение в файле video.c строка

5.51.4.6 SDL_Window* window

окно SDL

См. определение в файле video.c строка

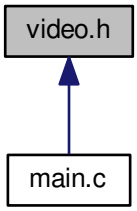
5.52 video.c

```
00001
00009 #include <stdio.h>
00010 #include <SDL.h>
00011
00012 #include "types.h"
00013 #include "timer.h"
00014 #include "key.h"
00015 #include "mouse.h"
00016
00017 #define WINDOW_WIDTH 640
00018 #define WINDOW_HEIGHT 400
00019 #define SCREEN_WIDTH 320
00020 #define SCREEN_HEIGHT 200
00021 #define FPS 40
00023 SDL_Window *window;
00024 SDL_Renderer *renderer;
00025 SDL_Surface *screen;
00026 long current_time;
00027 byte *video_buffer;
00028 long time_step = 1000 / FPS;
00034 void video_init()
00035 {
00036   SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO);
00037   window = SDL_CreateWindow("alis", SDL_WINDOWPOS_UNDEFINED, SDL_WINDOWPOS_UNDEFINED,
      WINDOW_WIDTH, WINDOW_HEIGHT, SDL_WINDOW_SHOWN);
00038   if (window == NULL) {
00039     printf("Cannot create window\n");
00040     exit(1);
00041   }
00042   renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED |
      SDL_RENDERER_PRESENTVSYNC);
00043   if (renderer == NULL) {
00044     printf("Cannot create renderer\n");
00045     exit(1);
00046   }
00047   current_time = SDL_GetTicks();
00048   screen = SDL_CreateRGBSurface(0, SCREEN_WIDTH, SCREEN_HEIGHT, 8, 0, 0, 0,
      0);
00049   SDL_SetHint(SDL_HINT_RENDER_SCALE_QUALITY, "1");
00050   SDL_LockSurface(screen);
00051   video_buffer = screen->pixels;
00052 }
00053
00059 void video_update()
00060 {
00061   long now = SDL_GetTicks();
00062   if (now - current_time < time_step)
00063     SDL_Delay(now - current_time);
00064   else
00065     current_time = now;
00066   timer_update();
00067   SDL_UnlockSurface(screen);
00068   SDL_RenderClear(renderer);
00069   SDL_Texture* t = SDL_CreateTextureFromSurface(renderer, screen);
00070   SDL_RenderCopy(renderer, t, NULL, NULL);
00071   SDL_DestroyTexture(t);
00072   SDL_RenderPresent(renderer);
00073   SDL_LockSurface(screen);
00074   video_buffer = screen->pixels;
00075 }
```

```
00076
00080 void video_close()
00081 {
00082   SDL_DestroyRenderer(renderer);
00083   SDL_DestroyWindow(window);
00084   SDL_Quit();
00085 }
00086
00090 void video_update_events()
00091 {
00092   SDL_Event e;
00093   int x, y, b;
00094
00095   if (SDL_PollEvent(&e)) {
00096     if (e.type == SDL_QUIT) {
00097       video_close();
00098       exit(0);
00099     } else if (e.type == SDL_KEYDOWN)
00100       set_key(e.key.keysym.scancode, e.key.keysym.sym, e.key.keysym.mod);
00101     else if (e.type == SDL_KEYUP)
00102       release_key(e.key.keysym.scancode);
00103     else if (e.type == SDL_MOUSEMOTION) {
00104       b = SDL_GetMouseState(&x, &y);
00105       mouse_event(x >> 1, y >> 1, b);
00106     } else if (e.type == SDL_MOUSEBUTTONDOWN) {
00107       SDL_GetMouseState(&x, &y);
00108       mouse_event(x >> 1, y >> 1, e.button.button);
00109     } else if (e.type == SDL_MOUSEBUTTONUP) {
00110       SDL_GetMouseState(&x, &y);
00111       mouse_event(x >> 1, y >> 1, 0);
00112     }
00113   }
00114 }
```

5.53 Файл video.h

Граф файлов, в которые включается этот файл:



Функции

- void video_init ()
- void video_update ()
- void video_update_events ()

5.53.1 Функции

5.53.1.1 void video_init ( )

Инициализация видео подсистемы. Создается окно и буфер экрана

См. определение в файле video.c строка

```
{
  SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO);
  window = SDL_CreateWindow("alis", SDL_WINDOWPOS_UNDEFINED, SDL_WINDOWPOS_UNDEFINED,
    WINDOW_WIDTH, WINDOW_HEIGHT, SDL_WINDOW_SHOWN);
  if (window == NULL) {
    printf("Cannot create window\n");
    exit(1);
  }
  renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED |
    SDL_RENDERER_PRESENTVSYNC);
  if (renderer == NULL) {
    printf("Cannot create renderer\n");
    exit(1);
  }
  current_time = SDL_GetTicks();
  screen = SDL_CreateRGBSurface(0, SCREEN_WIDTH, SCREEN_HEIGHT, 8, 0, 0, 0,
    0);
  SDL_SetHint(SDL_HINT_RENDER_SCALE_QUALITY, "1");
  SDL_LockSurface(screen);
  video_buffer = screen->pixels;
}
```

5.53.1.2 void video_update ( )

Обновление экрана. Делается задержка, чтобы соблюдать частоту кадров обновляется таймер, видео буфер выводится на экран.

См. определение в файле video.c строка

```
{
  long now = SDL_GetTicks();
  if (now - current_time < time_step)
    SDL_Delay(now - current_time);
  else
    current_time = now;
  timer_update();
  SDL_UnlockSurface(screen);
  SDL_RenderClear(renderer);
  SDL_Texture* t = SDL_CreateTextureFromSurface(renderer, screen);
  SDL_RenderCopy(renderer, t, NULL, NULL);
  SDL_DestroyTexture(t);
  SDL_RenderPresent(renderer);
  SDL_LockSurface(screen);
  video_buffer = screen->pixels;
}
```

5.53.1.3 void video_update_events ( )

Обработка событий клавиатуры и мыши

См. определение в файле video.c строка

```
{
  SDL_Event e;
  int x, y, b;

  if (SDL_PollEvent(&e)) {
    if (e.type == SDL_QUIT) {
      video_close();
      exit(0);
    } else if (e.type == SDL_KEYDOWN)
      set_key(e.key.keysym.scancode, e.key.keysym.sym, e.key.keysym.mod);
    else if (e.type == SDL_KEYUP)
      release_key(e.key.keysym.scancode);
    else if (e.type == SDL_MOUSEMOTION) {
      b = SDL_GetMouseState(&x, &y);
      mouse_event(x >> 1, y >> 1, b);
    } else if (e.type == SDL_MOUSEBUTTONDOWN) {
      SDL_GetMouseState(&x, &y);
      mouse_event(x >> 1, y >> 1, e.button.button);
    } else if (e.type == SDL_MOUSEBUTTONUP) {
      SDL_GetMouseState(&x, &y);
      mouse_event(x >> 1, y >> 1, 0);
    }
  }
}
```

5.54 video.h

```
00001 void video_init();
00002 void video_update();
00003 void video_update_events();
```

5.55 Файл view.c

Функции

- void view_update ()

5.55.1 Функции

5.55.1.1 void view_update ( )

См. определение в файле view.c строка

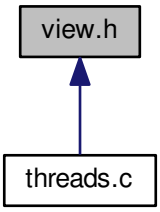
```
00002 {
00003 }
```

5.56 view.c

```
00001 void view_update()
00002 {
00003 }
```

5.57 Файл view.h

Граф файлов, в которые включается этот файл:



Функции

- void view_update ()

5.57.1 Функции

5.57.1.1 void view_update ( )

См. определение в файле view.c строка

```
00002 {
00003 }
```

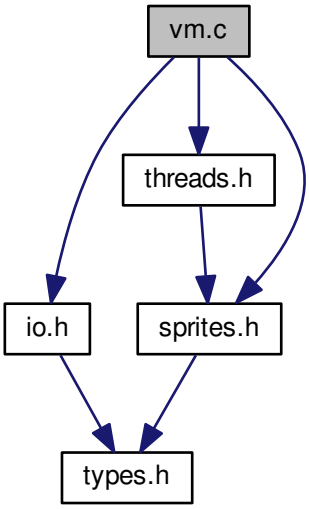
5.58 view.h

```
00001 void view_update();
```

5.59 Файл vm.c

Интерпретатор виртуальной машины

#include "io.h"
#include "threads.h"
#include "sprites.h"
Граф включаемых заголовочных файлов для vm.c:



Макросы

- #define FREE_MEM 1024
- #define CPU_SPEED 0xca
- #define HARDWARE_FLAG 0x84
- #define VIDEO_PORT 0x7d4
- #define MAX_STR 100

Функции

- void get_free ()
- void vm_init ()
- byte * vm_run (byte *ip)

Переменные

- short accum
- char get_string_buf [MAX_STR]
- char text_string_buf [MAX_STR]
- char store_string_buf [MAX_STR]
- char * get_string
- char * text_string
- char * store_string

5.59.1 Подробное описание

Интерпретатор виртуальной машины

Автор

alex <alex>

Дата

Wed Apr 6 18:07:27 2022

См. определение в файле vm.c

5.59.2 Макросы

5.59.2.1 #define CPU_SPEED 0xca

фиксированная скорость

См. определение в файле vm.c строка

5.59.2.2 #define FREE_MEM 1024

свободная память

См. определение в файле vm.c строка

5.59.2.3 #define HARDWARE_FLAG 0x84

флаги оборудования

См. определение в файле vm.c строка

5.59.2.4 #define MAX_STR 100

См. определение в файле vm.c строка

5.59.2.5 #define VIDEO_PORT 0x7d4

VGA порт

См. определение в файле vm.c строка

5.59.3 Функции

5.59.3.1 void get_free ( )

возвращает число свободных ресурсов

См. определение в файле vm.c строка

```
00030 {
00031   switch (accum) {
00032   case 0:  accum = FREE_MEM; break; // свободная память в кб
00033   case 1:  accum = FREE_MEM; break; // число свободной памяти потоков в кб
00034   case 2: // число свободных сценариев
00035     accum = io_num_free_files(); break;
00036   case 3: // число свободных потоков
00037     accum = threads_free_num(); break;
00038   case 4:
00039     accum = sprites_free_num(); break; // число свободных спрайтов
00040   case 'a':
00041   case 'A':
00042     accum = FREE_MEM; break; // свободных байт на диске A в кб
00043   case 'b':
00044   case 'B':
00045     accum = FREE_MEM; break; // свободных байт на диске B в кб
00046   case 'c':
00047   case 'C':
00048     accum = FREE_MEM; break; // свободных байт на диске C в кб
00049   case 'd':
00050   case 'D':
00051     accum = FREE_MEM; break; // свободных байт на диске D в кб
00052   default:
00053     accum = -1;
00054   }
00055 }
```

5.59.3.2 void vm_init ( )

См. определение в файле vm.c строка

```
00058 {
00059   get_string = get_string_buf;
00060   text_string = text_string_buf;
00061   store_string = store_string_buf;
00062 }
```

5.59.3.3 byte* vm_run (byte * ip)

См. определение в файле vm.c строка

```
00065 {
00066   video_update_events();
00067   return ip;
00068 }
```

5.59.4 Переменные

5.59.4.1 short accum

регистр аккумулятора

См. определение в файле vm.c строка

5.59.4.2 char* get_string

указатель строки для загрузки

См. определение в файле vm.c строка

5.59.4.3 char get_string_buf[MAX_STR]

буфер строки загрузки

См. определение в файле vm.c строка

5.59.4.4 char* store_string

указатель строки сохранения

См. определение в файле vm.c строка

5.59.4.5 char store_string_buf[MAX_STR]

буфер строки сохранения

См. определение в файле vm.c строка

5.59.4.6 char* text_string

указатель на строку для вывода

См. определение в файле vm.c строка

5.59.4.7 char text_string_buf[MAX_STR]

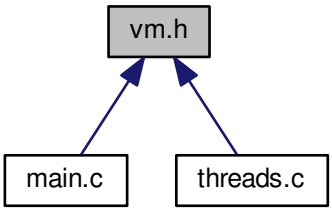
См. определение в файле vm.c строка

5.60 vm.c

```
00001
00008 #include "io.h"
00009 #include "threads.h"
00010 #include "sprites.h"
00011
00012 #define FREE_MEM 1024
00013 #define CPU_SPEED 0xca
00014 #define HARDWARE_FLAG 0x84
00015 #define VIDEO_PORT 0x7d4
00016 #define MAX_STR 100
00017
00018 short accum;
00019 char get_string_buf[MAX_STR];
00020 char text_string_buf[MAX_STR];
00021 char store_string_buf[MAX_STR];
00022 char *get_string;
00023 char *text_string;
00024 char *store_string;
00029 void get_free()
00030 {
00031   switch (accum) {
00032   case 0:  accum = FREE_MEM; break; // свободная память в кб
00033   case 1:  accum = FREE_MEM; break; // число свободной памяти потоков в кб
00034   case 2: // число свободных сценариев
00035     accum = io_num_free_files(); break;
00036   case 3: // число свободных потоков
00037     accum = threads_free_num(); break;
00038   case 4:
00039     accum = sprites_free_num(); break; // число свободных спрайтов
00040   case 'a':
00041   case 'A':
00042     accum = FREE_MEM; break; // свободных байт на диске A в кб
00043   case 'b':
00044   case 'B':
00045     accum = FREE_MEM; break; // свободных байт на диске B в кб
00046   case 'c':
00047   case 'C':
00048     accum = FREE_MEM; break; // свободных байт на диске C в кб
00049   case 'd':
00050   case 'D':
00051     accum = FREE_MEM; break; // свободных байт на диске D в кб
00052   default:
00053     accum = -1;
00054   }
00055 }
00056
00057 void vm_init()
00058 {
00059   get_string = get_string_buf;
00060   text_string = text_string_buf;
00061   store_string = store_string_buf;
00062 }
00063
00064 byte *vm_run(byte *ip)
00065 {
00066   video_update_events();
00067   return ip;
00068 }
```

5.61 Файл vm.h

Граф файлов, в которые включается этот файл:



Функции

- void vm_init ()
- byte * vm_run (byte *ip)

5.61.1 Функции

5.61.1.1 void vm_init ( )

См. определение в файле vm.c строка

```
00058 {
00059   get_string = get_string_buf;
00060   text_string = text_string_buf;
00061   store_string = store_string_buf;
00062 }
```

5.61.1.2 byte* vm_run (byte * ip)

См. определение в файле vm.c строка

```
00065 {
00066   video_update_events();
00067   return ip;
00068 }
```

5.62 vm.h

```
00001 void vm_init();
00002 byte *vm_run(byte *ip);
```

Предметный указатель

accum
    vm.c, 85
anim_count
    resource_table_t, 10
anim_table
    resource_table_t, 10

BLANCPC_HEADER
    blancpc.c, 20
BLANCPC_SIZE
    blancpc.c, 20
BLANCPC_USIZE
    blancpc.c, 20
BLANCPC
    blancpc.c, 20
blancpc.c, 19
    BLANCPC_HEADER, 20
    BLANCPC_SIZE, 20
    BLANCPC_USIZE, 20
    BLANCPC, 20
    blancpc_buffer, 21
    blancpc_data, 21
    buffer, 21
    load_blancpc, 21
blancpc.h, 22
    load_blancpc, 23
blancpc_buffer
    blancpc.c, 21
blancpc_data
    blancpc.c, 21
bool
    types.h, 73
buffer
    blancpc.c, 21
byte
    types.h, 73

CHANNEL_OFF
    sound.c, 52
CPU_SPEED
    vm.c, 83
call_sp
    thread_t, 14
channel_num
    sound.c, 54
channel_num_max_priority
    sound.c, 54
control
    io_header_t, 7
current_channel
    sound.c, 54
current_thread
    threads.c, 65
current_time
    video.c, 77
cursor_x
    text.c, 62
cursor_y
    text.c, 62

data_size
    io_header_t, 7
dword
    types.h, 73

f1
    sound_header_t, 12
f4
    sound_header_t, 12
FILE_APPEND
    file.h, 28
FILE_CREATE
    file.h, 28
FPS
    video.c, 75
FREE_MEM
    vm.c, 83
file.c, 23
    file_close, 24
    file_exists, 24
    file_name, 27
    file_open, 25
    file_read, 25
    file_write, 26
    handle, 27
    uppercase, 26
file.h, 28
    FILE_APPEND, 28
    FILE_CREATE, 28
    file_close, 29
    file_exists, 29
    file_open, 29
    file_read, 30
file_close
    file.c, 24
    file.h, 29
file_exists
    file.c, 24

file.h, 29
file_name
file.c, 27
file_open
file.c, 25
file.h, 29
file_read
file.c, 25
file.h, 30
file_write
file.c, 26
flags
sound_channel_t, 11
thread_t, 14
frame_num
thread_t, 15
timer.c, 72
free_sprites
sprites.c, 59
frequency
sound_channel_t, 11
frequency_gain
sound_channel_t, 11

gain
sound_channel_t, 11
get_free
vm.c, 84
get_key
key.c, 38
key.h, 41
get_key_mod
key.c, 38
key.h, 41
get_string
vm.c, 85
get_string_buf
vm.c, 85
global_init
main.c, 44

HARDWARE_FLAG
vm.c, 83
handle
file.c, 27

id
io_header_t, 8
image_count
resource_table_t, 10
image_table
resource_table_t, 10
io.c, 31
io_num_free_files, 32
io_unpack_sound, 32
loaded_io_files, 33
SOUND_TYPE1, 32
SOUND_TYPE2, 32
sound_offset_table, 33
total_io_files, 33
io.h, 35
io_num_free_files, 35
io_unpack_sound, 35
io_file
thread_t, 15
io_header_t, 7
control, 7
data_size, 7
id, 8
key_entry, 8
main_entry, 8
message_entry, 8
msg_size, 8
resources, 8
stack_size, 8
u1, 8
u2, 9
u4, 9
u5, 9
version, 9
io_num_free_files
io.c, 32
io.h, 35
io_unpack_sound
io.c, 32
io.h, 35
ip
thread_t, 15

key.c, 37
get_key, 38
get_key_mod, 38
key_mod, 39
key_symbol, 39
release_key, 38
set_key, 38
wait_for_key, 39
key.h, 41
get_key, 41
get_key_mod, 41
release_key, 41
set_key, 42
wait_for_key, 42
key_entry
io_header_t, 8
key_mod
key.c, 39
key_symbol
key.c, 39

length
sound_channel_t, 11
load_blancpc
blancpc.c, 21
blancpc.h, 23
loaded_io_files
io.c, 33
loaded_threads

threads.c, 65

MAX_STR
vm.c, 83
MAX_VOLUME
sound.c, 52
main
main.c, 44
main.c, 43
global_init, 44
main, 44
main_entry
io_header_t, 8
max_threads
threads.c, 65
memory
thread_t, 15
message_entry
io_header_t, 8
mouse.c, 45
mouse_buttons, 47
mouse_draw_cursor, 46
mouse_event, 46
mouse_get, 46
mouse_init, 46
mouse_x, 47
mouse_y, 47
show_cursor, 47
mouse.h, 48
mouse_draw_cursor, 48
mouse_event, 48
mouse_get, 49
mouse_init, 49
mouse_buttons
mouse.c, 47
mouse_draw_cursor
mouse.c, 46
mouse.h, 48
mouse_event
mouse.c, 46
mouse.h, 48
mouse_get
mouse.c, 46
mouse.h, 49
mouse_init
mouse.c, 46
mouse.h, 49
mouse_x
mouse.c, 47
mouse_y
mouse.c, 47
msg_size
io_header_t, 8

NUM_CHANNELS
sound.c, 52
next
sprite_t, 13
threads_table_t, 17
no_save
threads.c, 66
num_frames
thread_t, 15

packed
sound_header_t, 12
pad
sound_header_t, 13
palette.c, 49
palette_update, 49
palette.h, 50
palette_update, 50
palette_update
palette.c, 49
palette.h, 50
priority
sound_channel_t, 11

README.md, 50
release_key
key.c, 38
key.h, 41
renderer
video.c, 77
resource_table_t, 9
anim_count, 10
anim_table, 10
image_count, 10
image_table, 10
sound_count, 10
sound_table, 10
resources
io_header_t, 8
running
thread_t, 15

SCREEN_HEIGHT
video.c, 75
SCREEN_WIDTH
video.c, 75
SOUND_TYPE1
io.c, 32
SOUND_TYPE2
io.c, 32
screen
video.c, 77
set_cursor_pos
text.c, 62
text.h, 63
set_key
key.c, 38
key.h, 42
show_cursor
mouse.c, 47
size
sound_header_t, 13
sound.c, 51
CHANNEL_OFF, 52

channel_num, 54
channel_num_max_priority, 54
current_channel, 54
MAX_VOLUME, 52
NUM_CHANNELS, 52
sound_channel_off, 52
sound_channels, 55
sound_find_channel_max_priority, 53
sound_init, 53
sound_process_channel, 53
sound_update, 54
synth_play, 54
sound.h, 56
sound_init, 57
sound_update, 57
sound_channel_off
sound.c, 52
sound_channel_t, 10
flags, 11
frequency, 11
frequency_gain, 11
gain, 11
length, 11
priority, 11
synth, 12
volume, 12
sound_channels
sound.c, 55
sound_count
resource_table_t, 10
sound_find_channel_max_priority
sound.c, 53
sound_header_t, 12
f1, 12
f4, 12
packed, 12
pad, 13
size, 13
type, 13
sound_init
sound.c, 53
sound.h, 57
sound_offset_table
io.c, 33
sound_process_channel
sound.c, 53
sound_table
resource_table_t, 10
sound_update
sound.c, 54
sound.h, 57
sprite_t, 13
next, 13
sprites
thread_t, 15
sprites.c, 57
free_sprites, 59
sprites_free_num, 59
sprites_set_translate, 59
sprites_translate, 59
sprites.h, 60
sprites_free_num, 61
sprites_set_translate, 61
sprites_translate, 61
sprites_free_num
sprites.c, 59
sprites.h, 61
sprites_set_translate
sprites.c, 59
sprites.h, 61
sprites_translate
sprites.c, 59
sprites.h, 61
stack_size
io_header_t, 8
store_string
vm.c, 85
store_string_buf
vm.c, 85
synth
sound_channel_t, 12
synth_play
sound.c, 54

THREAD_MSG
threads.h, 69
THREAD_NOMSG
threads.h, 69
THREAD_NOSTART3
threads.h, 69
text.c, 61
cursor_x, 62
cursor_y, 62
set_cursor_pos, 62
text.h, 62
set_cursor_pos, 63
text_string
vm.c, 85
text_string_buf
vm.c, 85
thread
threads_table_t, 17
thread_flag
threads.c, 66
thread_t, 14
call_sp, 14
flags, 14
frame_num, 15
io_file, 15
ip, 15
memory, 15
num_frames, 15
running, 15
sprites, 15
threads.c, 63
current_thread, 65
loaded_threads, 65

max_threads, 65
no_save, 66
thread_flag, 66
threads_free_num, 64
threads_init, 64
threads_list, 66
threads_list_pos, 66
threads_run, 64
threads_table, 66
threads.h, 67
THREAD_MSG, 69
THREAD_NOMSG, 69
THREAD_NOSTART3, 69
threads_free_num, 69
threads_init, 69
threads_run, 69
threads_free_num
threads.c, 64
threads.h, 69
threads_init
threads.c, 64
threads.h, 69
threads_list
threads.c, 66
threads_list_pos
threads.c, 66
threads_run
threads.c, 64
threads.h, 69
threads_table
threads.c, 66
threads_table_t, 16
next, 17
thread, 17
time_step
video.c, 77
timer.c, 71
frame_num, 72
timer_update, 71
timer.h, 72
timer_update, 72
timer_update
timer.c, 71
timer.h, 72
total_io_files
io.c, 33
type
sound_header_t, 13
types.h, 73
bool, 73
byte, 73
dword, 73
word, 73

u1
io_header_t, 8
u2
io_header_t, 9
u4
io_header_t, 9
u5
io_header_t, 9
uppercase
file.c, 26

VIDEO_PORT
vm.c, 84
version
io_header_t, 9
video.c, 74
current_time, 77
FPS, 75
renderer, 77
SCREEN_HEIGHT, 75
SCREEN_WIDTH, 75
screen, 77
time_step, 77
video_buffer, 77
video_close, 76
video_init, 76
video_update, 76
video_update_events, 76
WINDOW_HEIGHT, 75
WINDOW_WIDTH, 75
window, 78
video.h, 79
video_init, 79
video_update, 80
video_update_events, 80
video_buffer
video.c, 77
video_close
video.c, 76
video_init
video.c, 76
video.h, 79
video_update
video.c, 76
video.h, 80
video_update_events
video.c, 76
video.h, 80
view.c, 81
view_update, 81
view.h, 81
view_update, 82
view_update
view.c, 81
view.h, 82
vm.c, 82
accum, 85
CPU_SPEED, 83
FREE_MEM, 83
get_free, 84
get_string, 85
get_string_buf, 85
HARDWARE_FLAG, 83
MAX_STR, 83

store_string, 85
store_string_buf, 85
text_string, 85
text_string_buf, 85
VIDEO_PORT, 84
vm_init, 84
vm_run, 84
vm.h, 87
vm_init, 87
vm_run, 87
vm_init
vm.c, 84
vm.h, 87
vm_run
vm.c, 84
vm.h, 87
volume
sound_channel_t, 12

WINDOW_HEIGHT
video.c, 75
WINDOW_WIDTH
video.c, 75
wait_for_key
key.c, 39
key.h, 42
window
video.c, 78
word
types.h, 73